

WorkshopPlus: Azure AI Platform and Services

Vector Databases

Conditions and terms of use

© Microsoft Corporation. All rights reserved.

You may use these training materials solely for your personal internal reference and non-commercial purposes. You may not distribute, transmit, resell or otherwise make these training materials available to any other person or party without express permission from Microsoft Corporation. URL's or other internet website references in the training materials may change without notice. Unless otherwise noted, any companies, organizations, domain names, e-mail addresses, people, places and events depicted in the training materials are for illustration only and are fictitious. No real association is intended or inferred. THESE TRAINING MATERIALS ARE PROVIDED "AS IS"; MICROSOFT MAKES NO WARRANTIES, EXPRESS OR IMPLIED IN THESE TRAINING MATERIALS¹

Learning Units covered in this Module



Vector Databases – basics, challenges



Cosmos DB - search



PostgreSQL - search



SQL DB - search



Objectives

After completing this Learning, you will be able to:

1	Understand.... ✓ fundamentals of vector search and how it differs from traditional keyword search
2	Identify.... ✓ when to use Cosmos DB NoSQL, Postgres Flex, or SQL DB and when to tap into AI Search for vector-based scenarios.
3	Apply.... ✓ full text search and BM25 reranking to improve relevance in hybrid search pipelines
4	Understand.... ✓ how DiskANN enables scalable and efficient approximate nearest neighbor search
5	Implement.... ✓ vector search capabilities using native support or extensions across Azure databases



Lesson 1 – Vector Databases

Text + vector search in databases

Search problems

Literal string filters only match exact letters

Wildcard searches do not understand context

Full text indexes can help, but still lack context

- Synonym lists are hard to maintain

Context matters

- Apple (fruit) and Apple (company) are totally different things

Non-text data is cumbersome to search + must be well-tagged

LLMs to the Rescue

Can turn Text into meaning

- Represented through numeric arrays (embeddings)

LLMs can take words/sentences/documents and outputs the “address” on a map

- Similar ideas land next to each other on the map

Help to understand context, not just spelling

- “Apple launches new iPhone” ≠ “apple pie recipe”

Language Flexibility

- “Car,” “automobile,” “EV,” and even the Spanish “coche” cluster together automatically

One approach works for all data types

- Text captions, product reviews, support tickets—anything convertible to text can be embedded the same way
- Vectors from images or audio (via specialized encoders) share the same kind of searchable format

Using Vector Indexes in Azure Databases and/or Azure AI Search

Vector Indexes in Azure databases are preferred when:

- You have structured or semi-structured **operational data** (e.g. chat history, customer profiles, business transactions etc.) in that database
- Simplified architecture for **single source of truth** combining **vector similarity search inline with database queries**. No need to synchronize separated database and vector index
- The workload benefits from mission-critical **OLTP database characteristics** (e.g. stronger guarantees for availability and performance)



NoSQL semi-structured data



Azure Cosmos DB

PostgreSQL familiarity
Relational data



Azure Database
for PostgreSQL

SQL Server familiarity
Relational data



Azure SQL DB

Azure AI Search is preferred when:

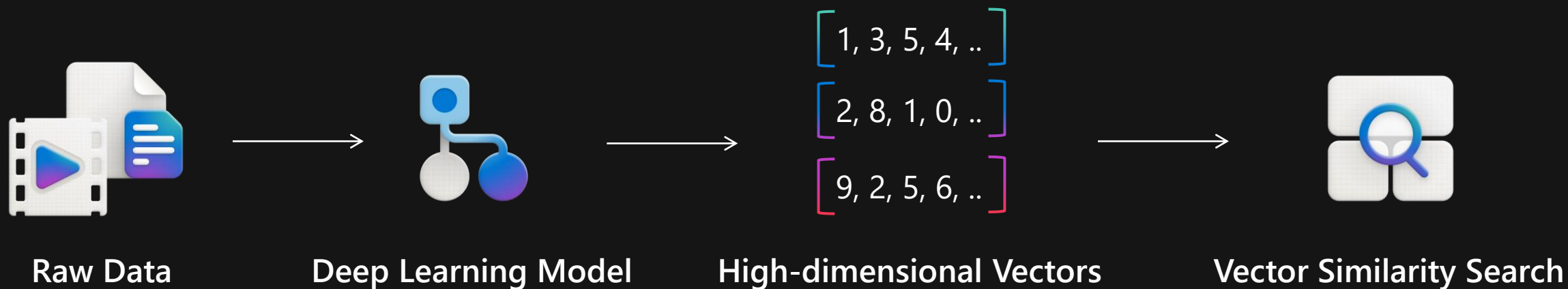
- You need to index structured/unstructured data (e.g. images, docx, PDFs) from a **variety of internal and external data sources**
- Your application requires **state-of-the-art search technology** for higher search quality (e.g. hybrid full-text/vector search, fuzzy, autocomplete, semantic re-ranking, multi-language, metafiltering)
- The workload requires **multi-modal search** and/or multi-modal embeddings to perform OCR, image analysis, translation, etc.
- You're building a **Bing-like search experience** in a custom application



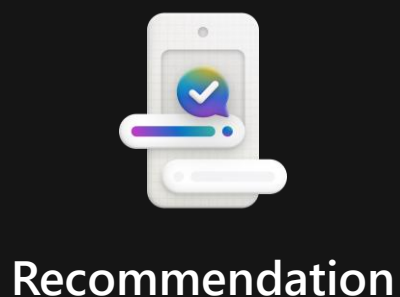
Azure AI Search



Vectors: Important in the AI Era



Vector similarity search empowers Generative AI apps

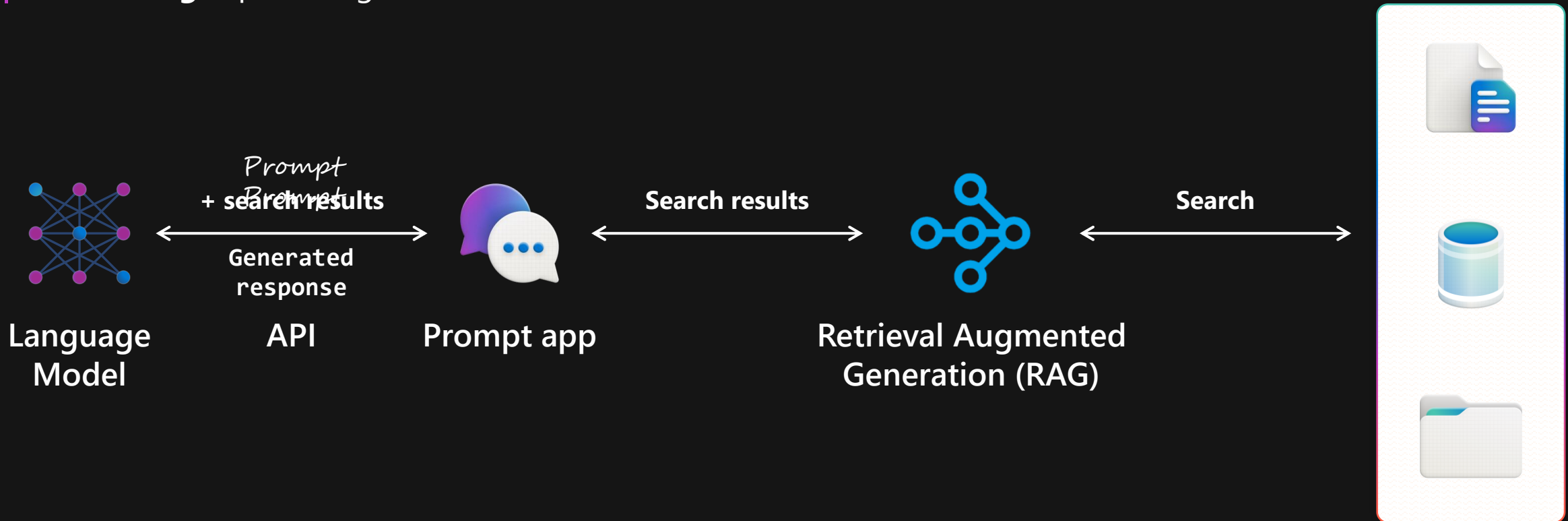


...

Let's get grounded on prompts

Fine tuning = customizing a trained model

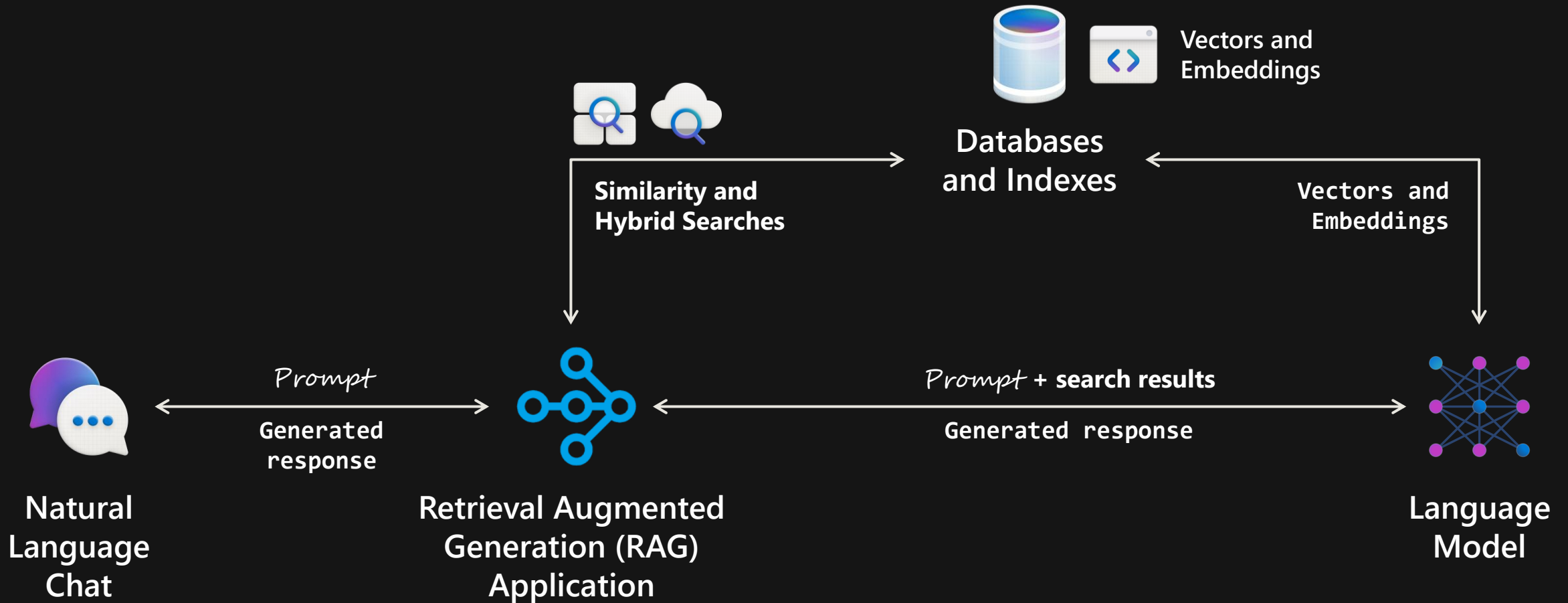
Grounding = providing additional context to a trained model



Prompt Engineering

Better prompt = better responses

Building Generative AI applications with databases



Vectors and Embeddings

Feature Vector

Ordered array of numbers typically created by a human to train a model
[Height, Weight, Age, Fur Length, Energy Level]

Embedding

Vector generated by a model that has semantic meaning

dog: [0.9, 0.3, 0.2, ...]

Image of a dog: [0.9, 0.3, 0.2, ...]

puppy: [0.88, 0.33, 0.21, ...]

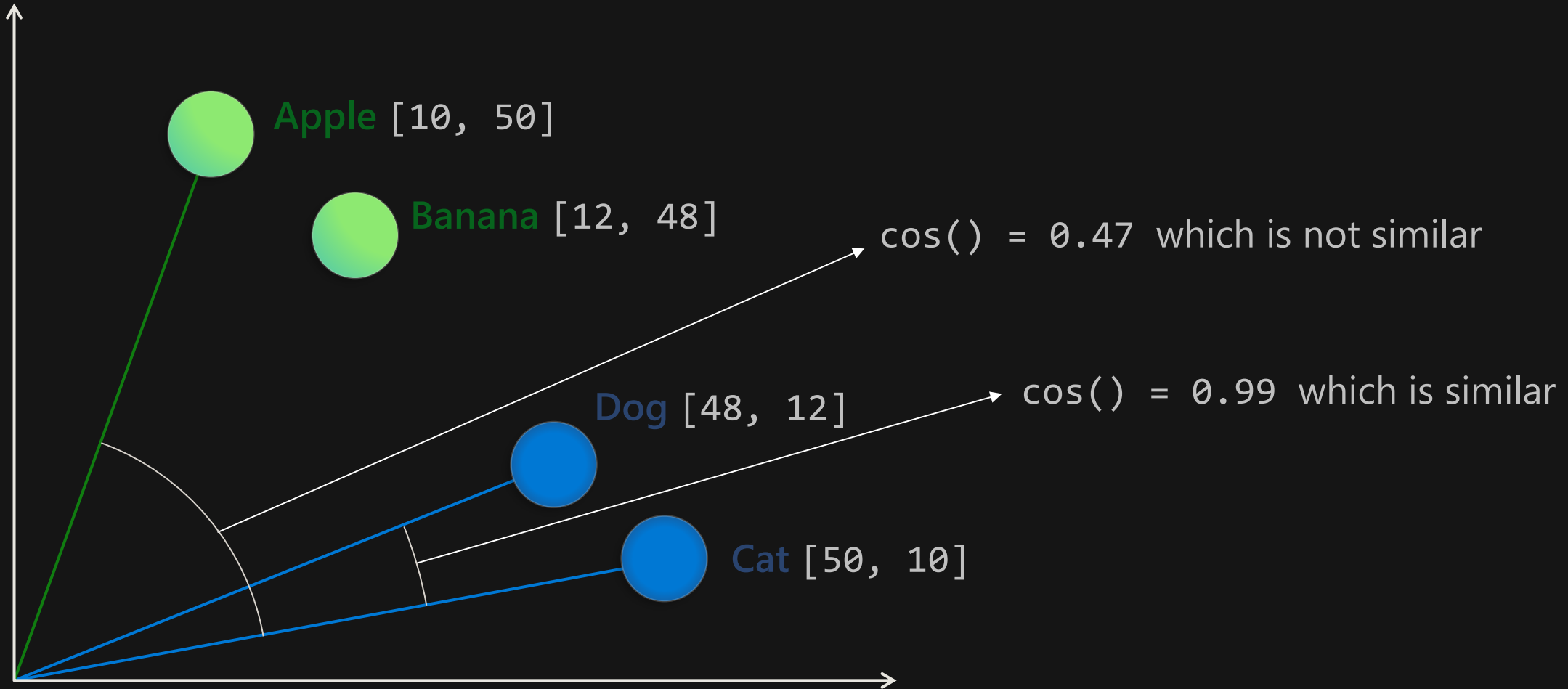
"I took the dog for a walk": [1.5, -0.8, 2.1, ...]

"dog"

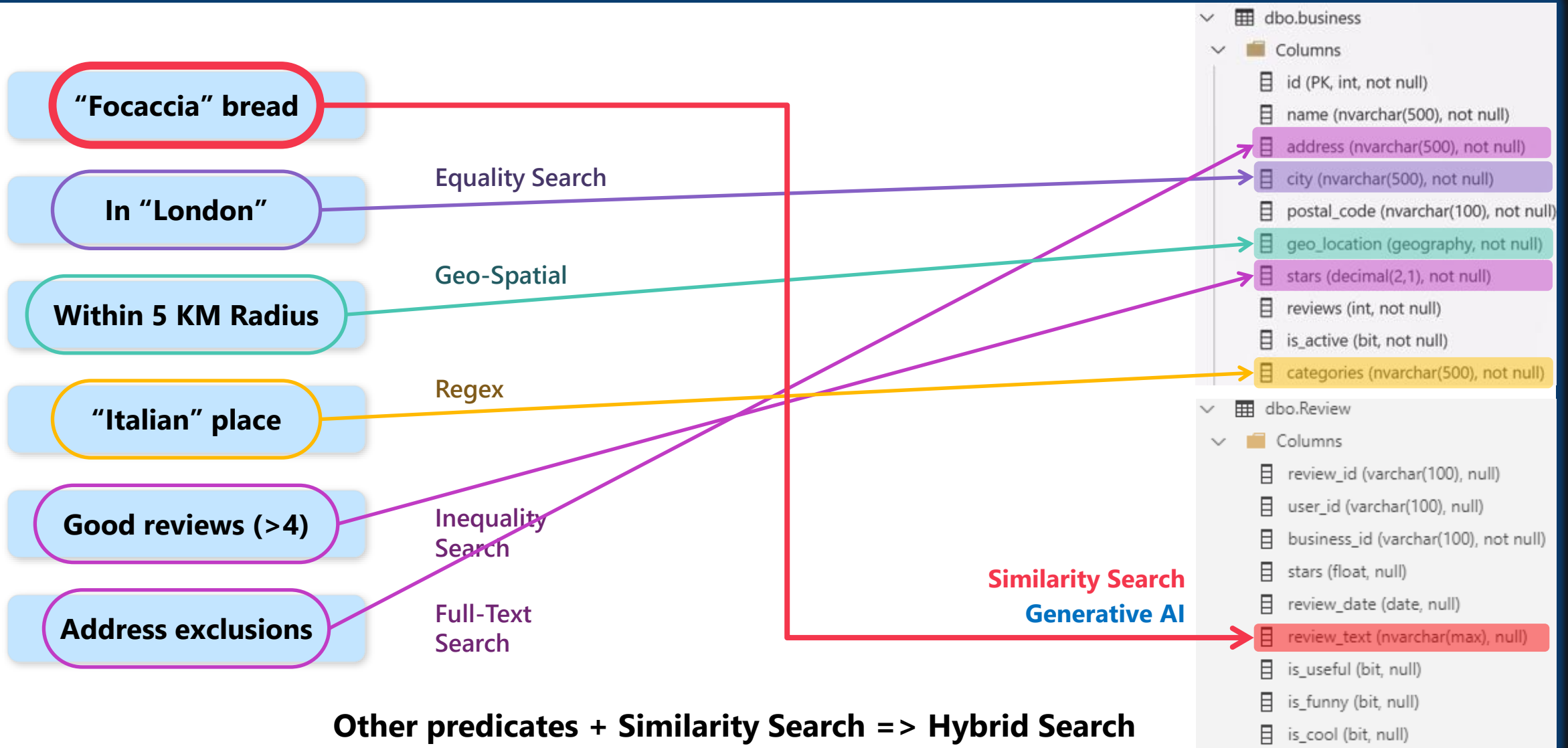


🔊 "dog"

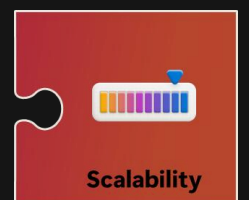
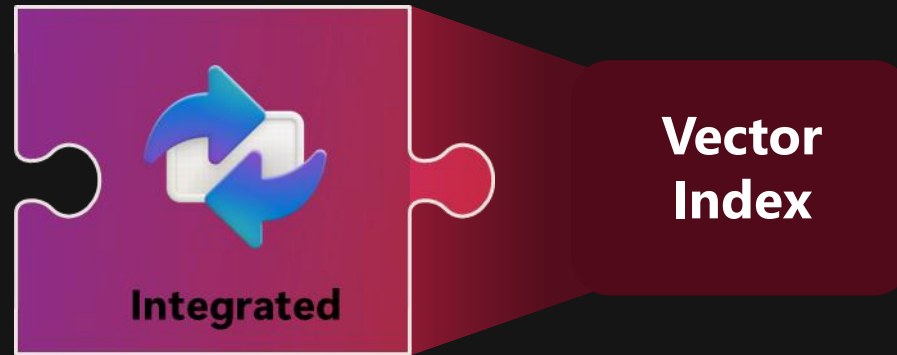
Similarity Searching using Cosine



Hybrid Search : Finding the Best Focaccia Bread in town



Selecting your Azure Database technology to run your AI apps at any scale



Approaches to Vector Search

kNN

Nearest Neighbor search
Brute force / Exact search

- Compare query vector with every vector in your DB
- 100% recall/accuracy
- Slow, scales linearly

No
Index

Flat
Index

Column Store
Index

ANN

Approximate Nearest Neighbor search

- Use indexes, clustering, graphs, etc. to lower # of compares
- Popular types: HNSW, IVF, ScANN, DiskANN

Quantized
Flat Index

HNSW
Index

DiskANN
Index

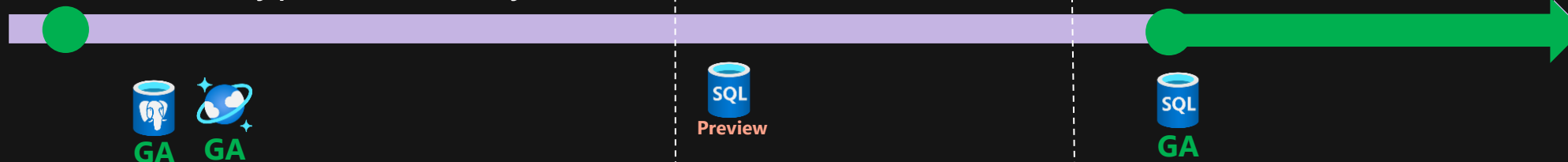
IVF
Index

Recently

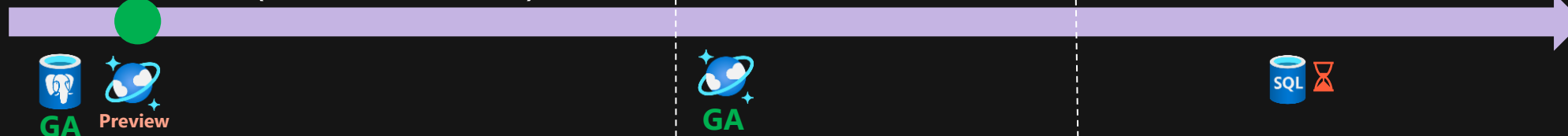
Currently

Near Future

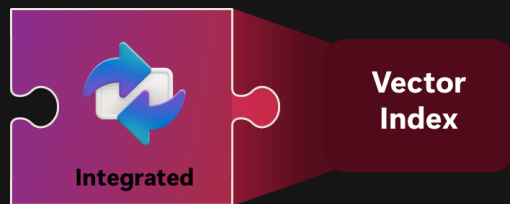
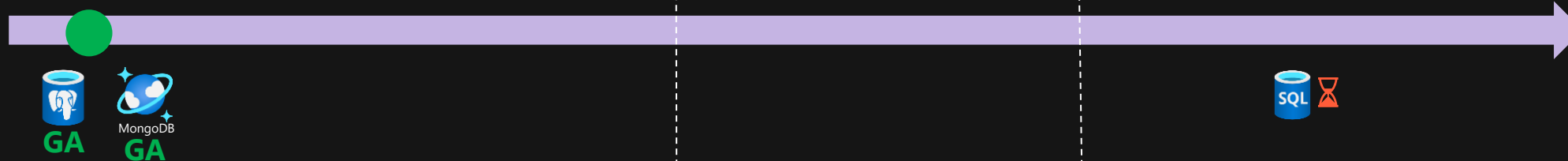
Vector Data Type and Query



Vector Index (IVFFLAT / Flat)



Vector Index with HNSW



Azure Databases with DiskANN built-in

Vector compression

Large Vectors

{ D1, D2, D3, D4, D5, ..., D99, D100 }



Quantization



Compressed Vectors

{ D1, D2 ..., D10 }

Storage and graph construction

RAM

Compressed vectors

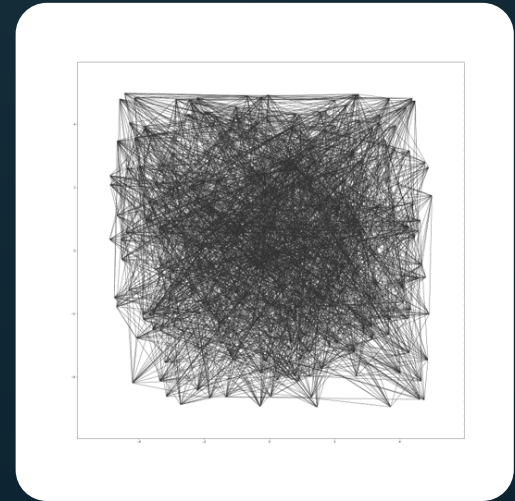


SSD

Full vectors + graph



Algorithms

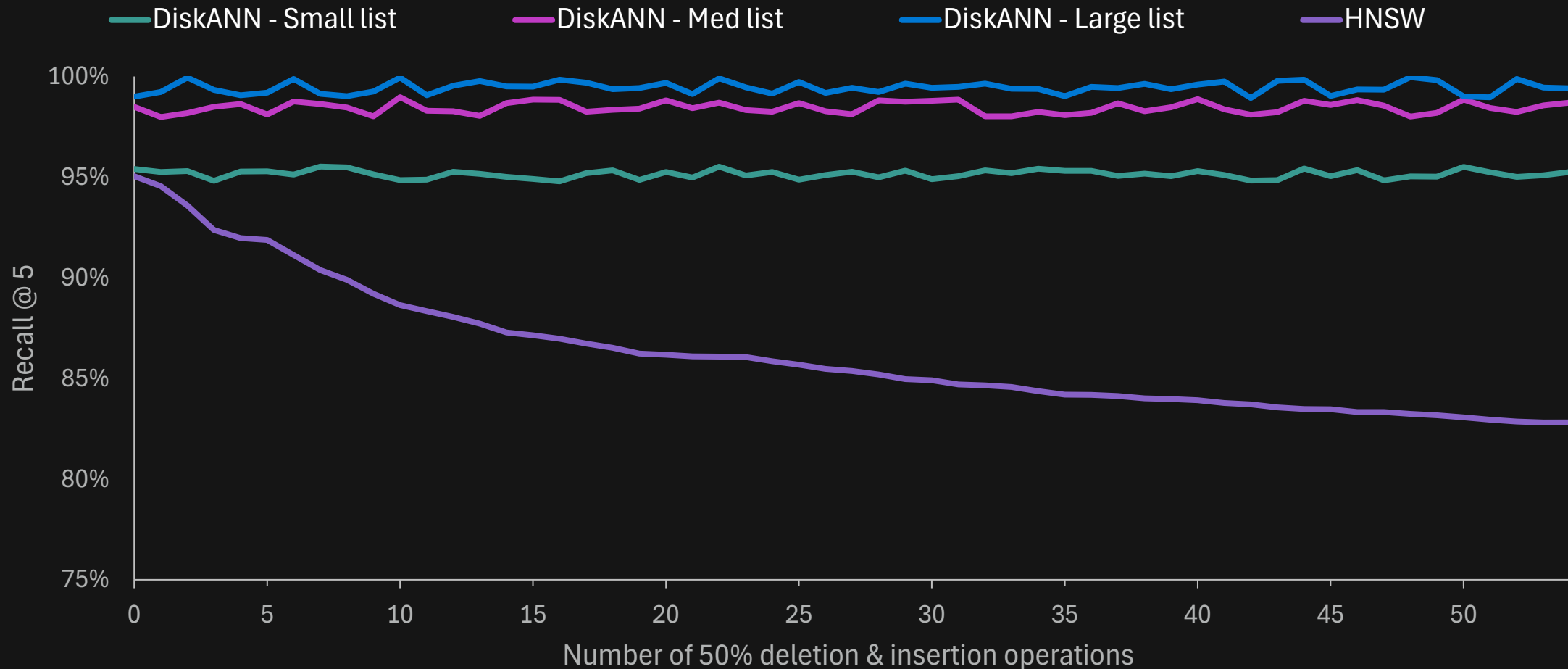


Unlimited scale

Low latency

Robust to data changes

DiskANN accuracy persists with many updates



Recently

Currently

Near Future

Vector Data Type and Query



Vector Index (IVFFLAT / Flat)

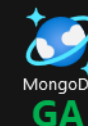
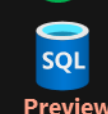
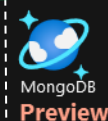
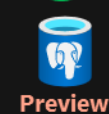


Vector Index with HNSW



Index
Feature
Parity

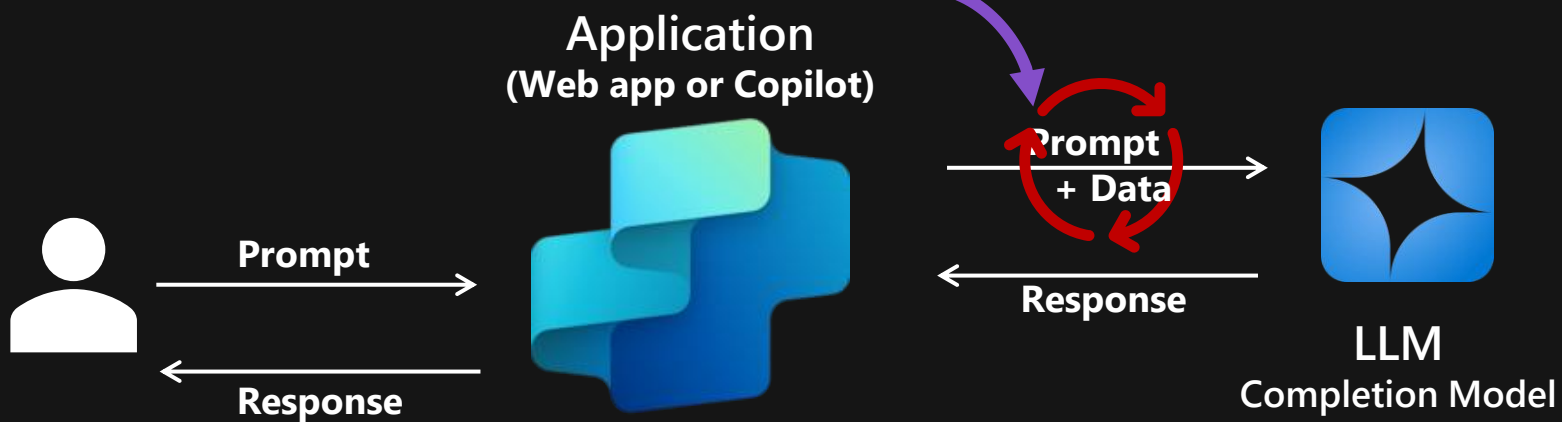
Vector Index with DiskANN



How does RAG work?

Grounding

providing additional context to a trained model
= improved performance



System Prompt

You are a helpful weather agent that always provides fun, friendly responses and occasionally includes some fun facts or jokes in your answers.

Today's weather in the following cities:
Atlanta 48° 63°
Miami 72° 78°
Seattle 32° 38°

Additional Data or Context

Don't use weather data from any other source, if you don't have data say "I don't know"

Restrictive Prompt clause

User Prompt

What is the weather in Seattle



For today in **Seattle**, it's 32°F in the morning and warming up to 38°F later. 🧊 Definitely a chilly one—bundle up! Maybe grab a nice cup of Seattle's famous coffee to stay warm. ☕

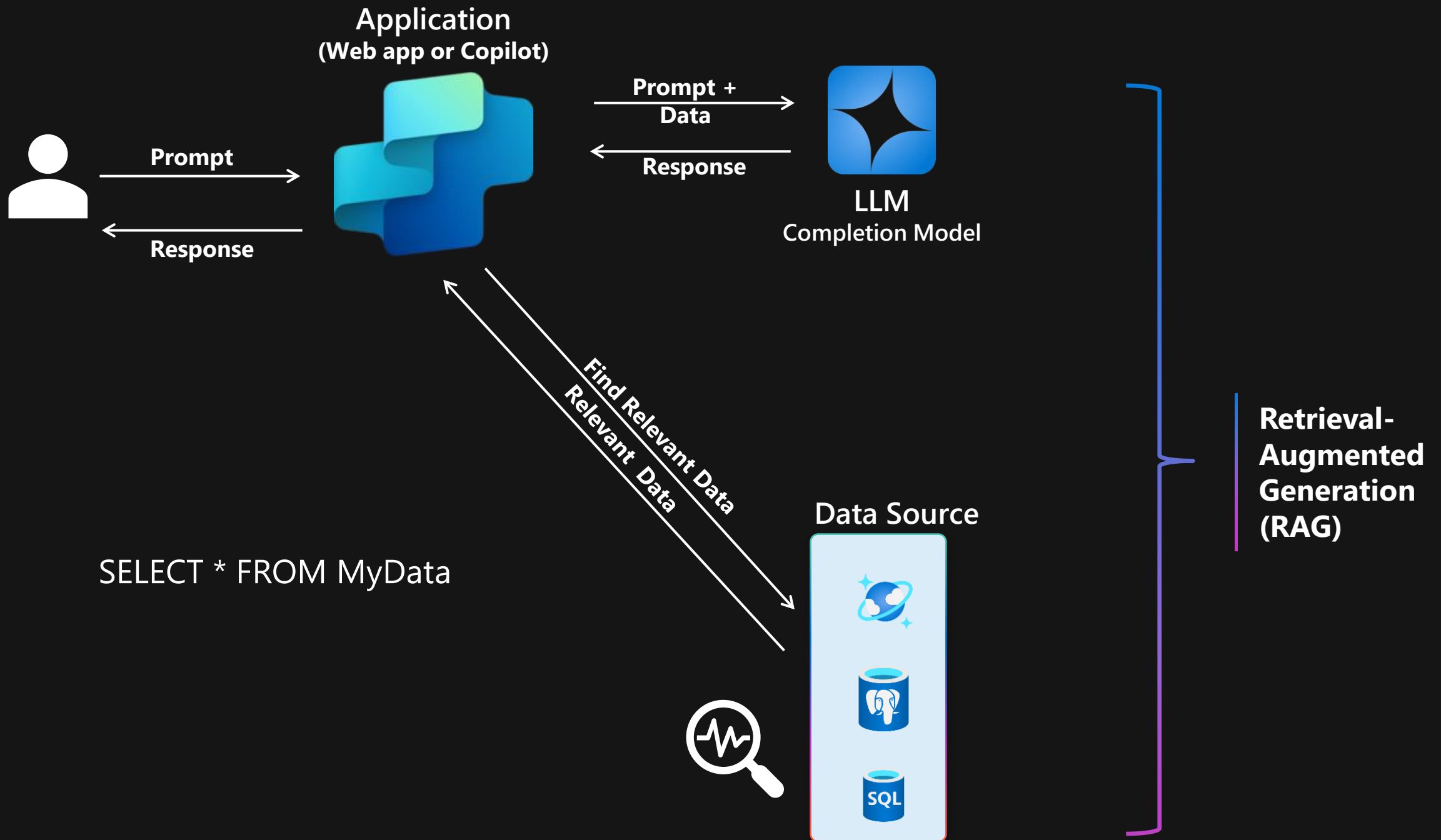
Fun fact: Seattleites buy more sunglasses per capita than any other U.S. city. Guess all those cloudy days make people *extra* excited when the sun peaks out! 😎☁️

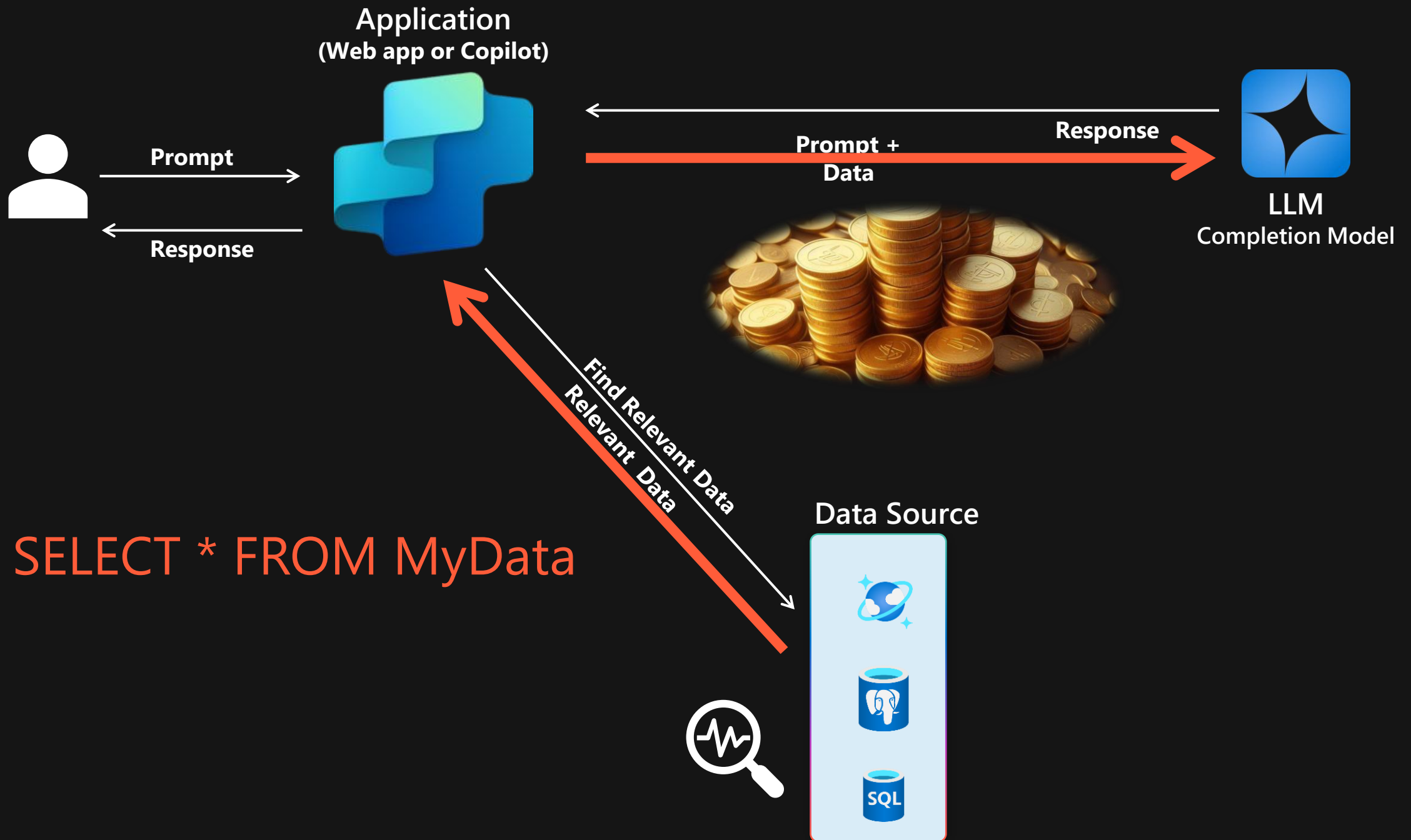
What is the weather in Chicargo

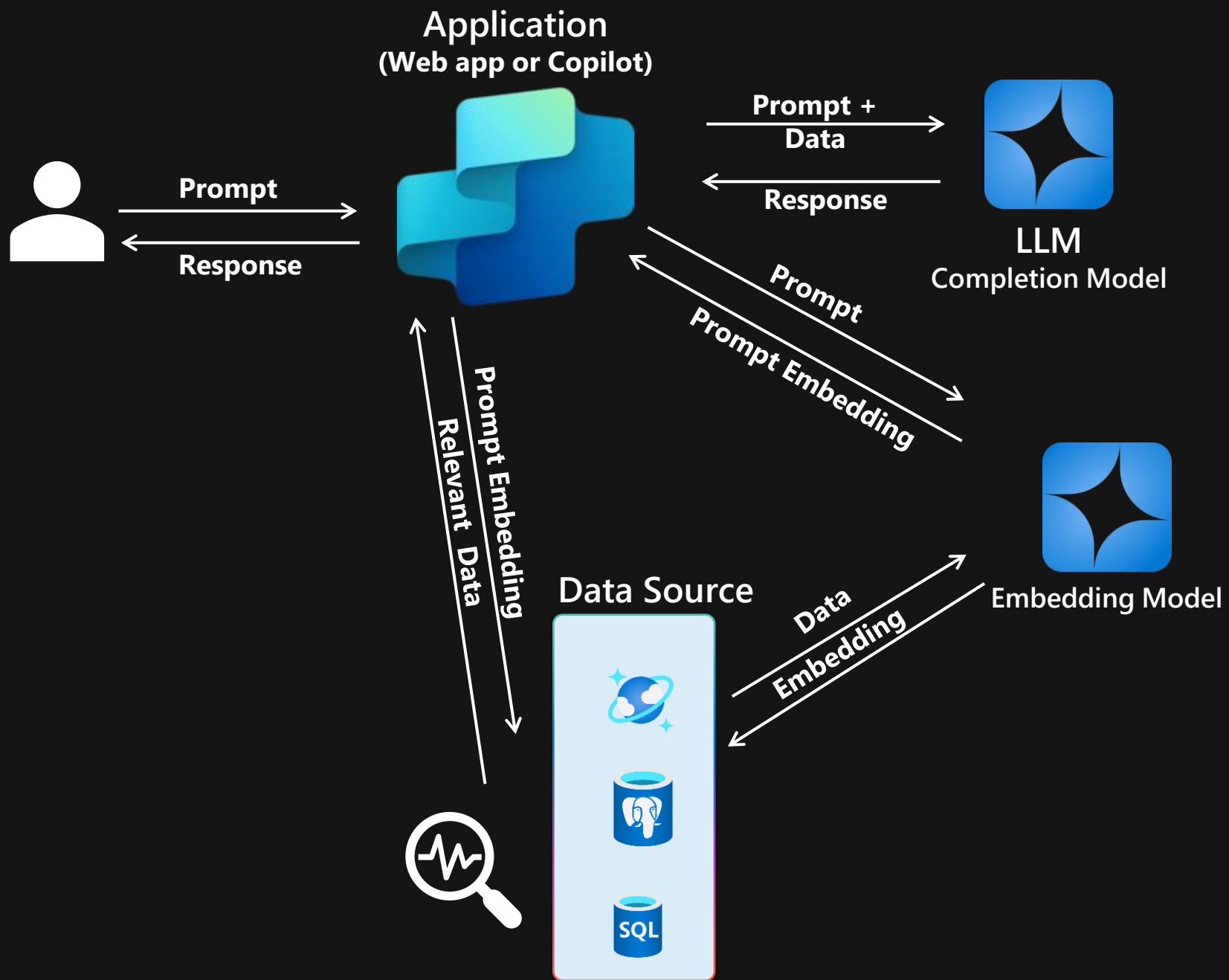


I don't know the weather in **Chicargo** today. But if you're there, I'd recommend layering up—Chicago is called the "**Windy City**" for a reason! 🌬️😬

Let me know if you need a weather joke while you wait! ☁️😄







RAG + chat history

What is the largest Mammal?

User
Prompt



The largest mammal is the **blue whale** (*Balaenoptera musculus*).

- **Length:** Up to 100 feet (30 meters)
- **Weight:** Up to 200 tons (180,000 kilograms)

It's not only the largest living mammal but also the largest animal known to have ever existed on Earth.

What is the smallest?

User
Prompt



Could you clarify what you mean by "the smallest"? Are you referring to:

- The smallest **animal**?
- The smallest **country**?
- The smallest **unit of measurement**?
- The smallest **known particle**?
- Something else?

Let me know what you're looking for so I can give you a precise answer.

What is the largest Mammal?

User
Prompt



The largest mammal is the **blue whale** (*Balaenoptera musculus*).

- **Length:** Up to 100 feet (30 meters)
- **Weight:** Up to 200 tons (180,000 kilograms)

It's not only the largest living mammal but also the largest animal known to have ever existed on Earth.

What is the smallest?

User
Prompt



The **smallest mammal** by mass is the **bumblebee bat** (*Craseonycteris thonglongyai*), also known as **Kitti's hog-nosed bat**.

- **Weight:** Around 2 grams (0.07 ounces)
- **Length:** About 1.1–1.3 inches (2.8–3.3 cm)

It lives in limestone caves along rivers in Thailand and Myanmar. Despite its tiny size, it's a fully functional bat, capable of flight and echolocation.

What is the largest Mammal?

User
Prompt



The largest mammal is the **blue whale** (*Balaenoptera musculus*).

- **Length:** Up to 100 feet (30 meters)
- **Weight:** Up to 200 tons (180,000 kilograms)

It's not only the largest living mammal but also the largest animal known to have ever existed on Earth.

Conversation
Context

+

User
Prompt

What is the smallest?

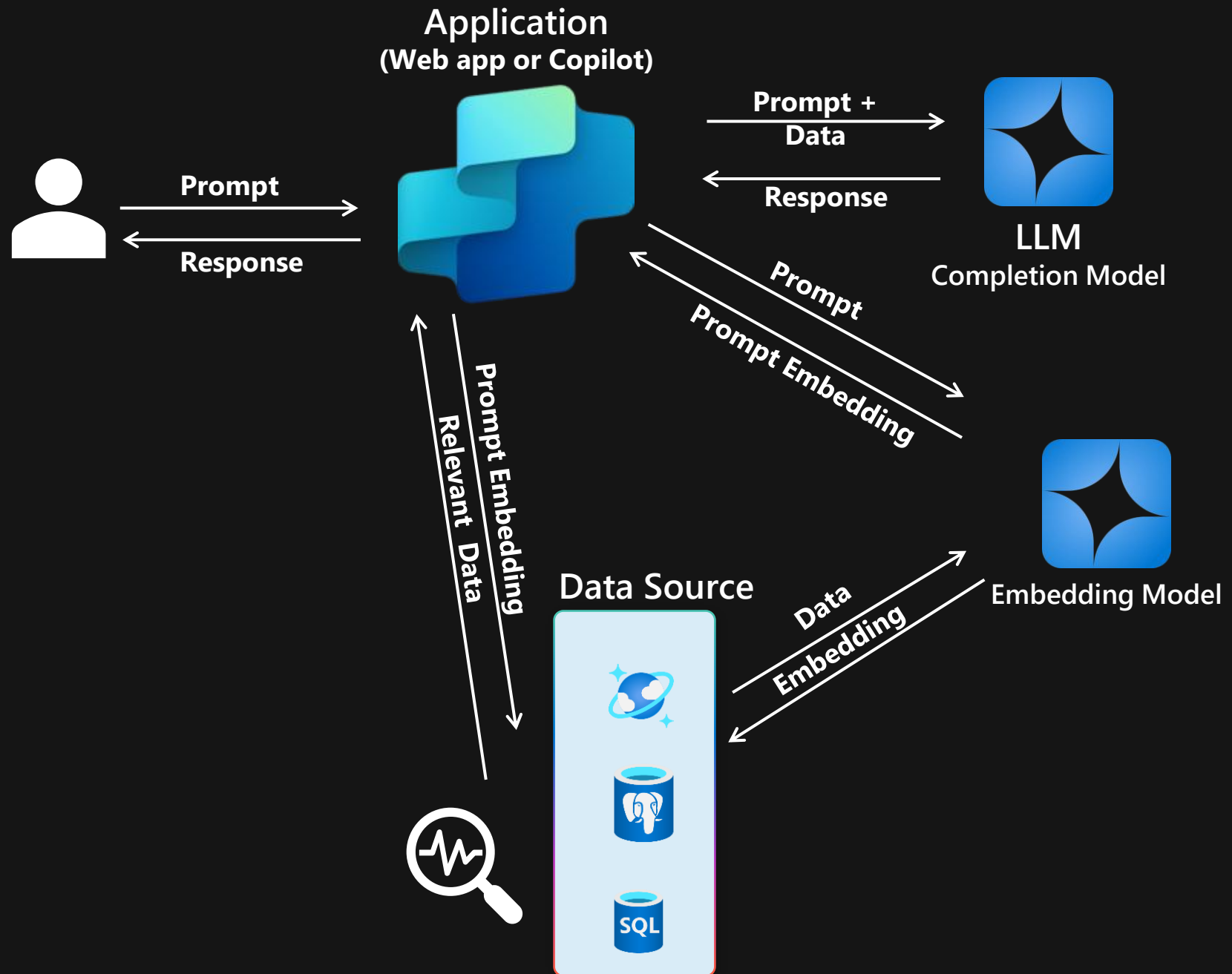
User
Prompt

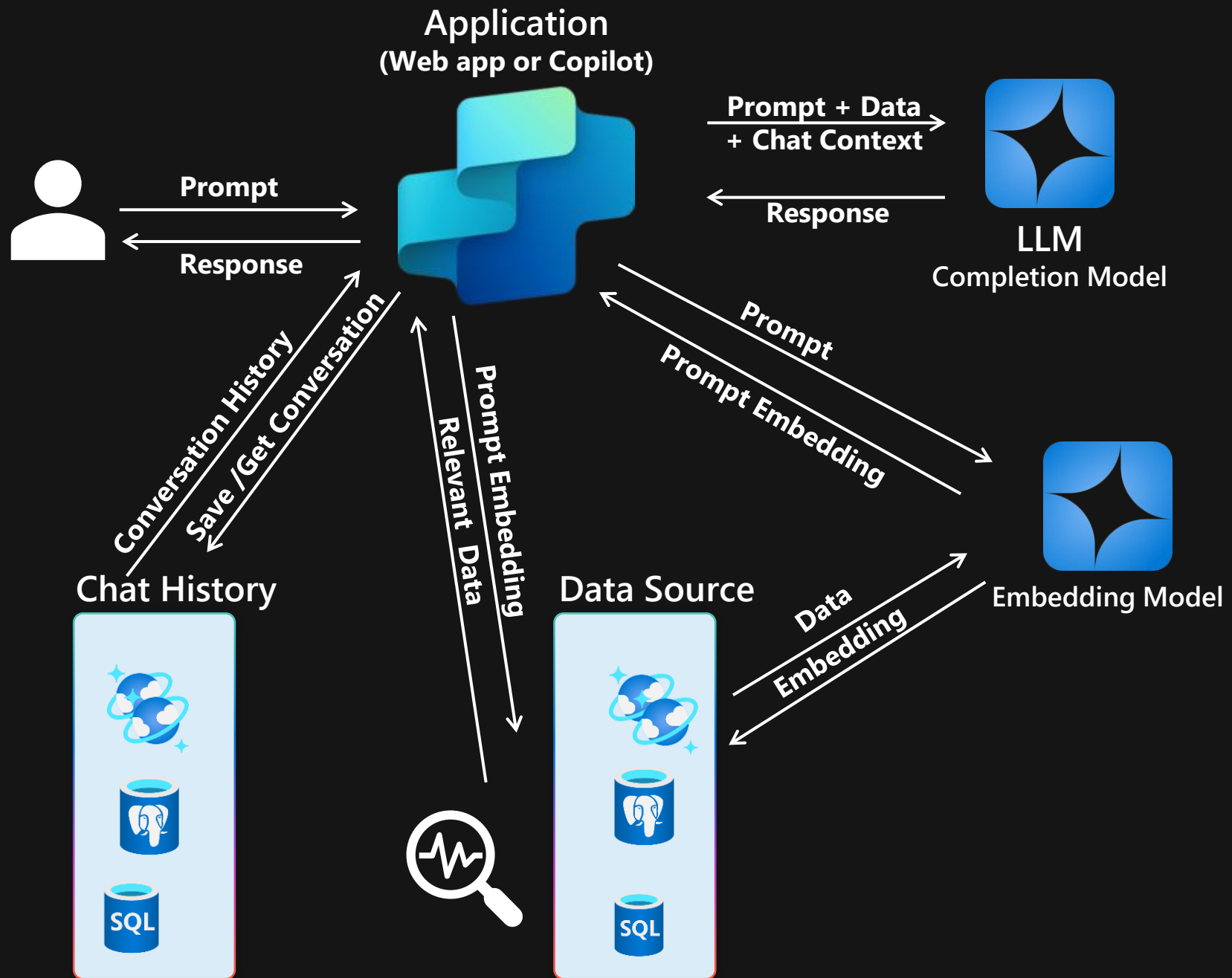


The **smallest mammal** by mass is the **bumblebee bat** (*Craseonycteris thonglongyai*), also known as **Kitti's hog-nosed bat**.

- **Weight:** Around 2 grams (0.07 ounces)
- **Length:** About 1.1–1.3 inches (2.8–3.3 cm)

It lives in limestone caves along rivers in Thailand and Myanmar. Despite its tiny size, it's a fully functional bat, capable of flight and echolocation.





Agents – Introduction

What is an AI Agent?

Definitions from the web:

1

"An **AI agent** is a software entity designed to perform tasks autonomously or semi-autonomously by receiving input, processing information, and taking actions to achieve specific goals." – **Semantic Kernel**

2

"**Agentic AI** refers to artificial intelligence capable of making decisions, planning, and adapting to new information in real time."
– **GitHub AI/ML blog**

3

"An **agent** is a system that uses an LLM to decide the control flow of an application"
– **LangGraph**

Common features of AI Agents



Planning

AI agents can create step-by-step plans and can use feedback to improve refine future outputs.



Memory

Agents rely on short-term memory for immediate prompts and leverage long-term data retention—often via retrieval-augmented generation (RAG)—for broader context and recall.



Perception

AI agents can retrieve and process information making them more interactive and contextually aware.



Tool Use

Agents can run functions or call external APIs for information or to perform actions.

AI Agents are a lot like “intelligent” functions



Take Inputs and Produce Outputs:

Just as functions receive parameters and return results, AI agents process prompts or data and then generate responses or actions.



Encapsulate Logic:

Functions contain code that executes a specific task; AI agents encapsulate reasoning steps and decision-making processes.



Modular and Reusable:

Both functions and AI agents can be “called” repeatedly in different parts of an application, each time performing their specialized task.



Composable:

They can be chained or orchestrated with other agents/functions, enabling complex workflows from simpler building blocks.

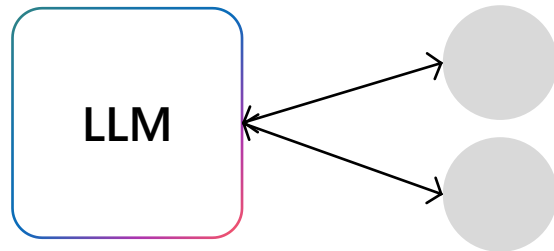
Core terms

Agent:	An AI system that can answer questions and automate processes for users. Agents can determine which functions will meet a user's goal, and then call those functions on the user's behalf.
Environment:	The external context or world in which an agent operates. It includes everything that an agent can perceive, interact with, or affect.
Perception:	The process by which an agent gathers information about its environment using data from sensors, databases, etc.
Action:	The effect produced by an agent on its environment as a result of executing a decision or policy. Actions can be physical (e.g., moving a robot's arm) or symbolic (e.g., sending a message).
Memory:	Pieces of information, data, interactions, or transactions that occurred previously by the agent(s). These may be useful later in time or for offline analysis.
State:	The current configuration or condition of an agent and its environment at a particular point in time. It represents all relevant information needed to make decisions.
Function / Tool:	Enabling the agent to interact with external systems, code, etc.
Multi-Agent Systems (MAS):	Systems composed of multiple interacting agents, each with its own goals, capabilities, and knowledge. MAS study the dynamics of agent interactions and their collective behavior.

Common Agent Architectures

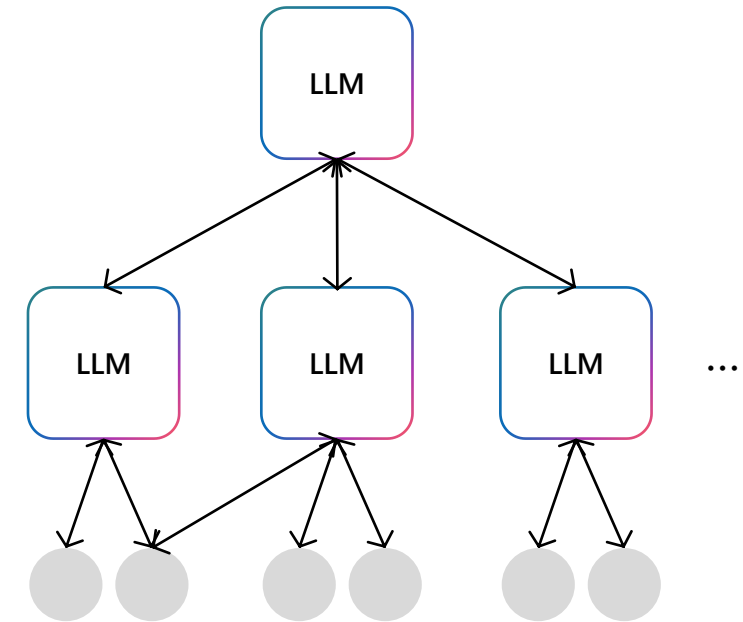
Single-Agent

Functions (Tools)



Router (Supervisor, Planner)

Functions (Tools)



Levels of autonomy in LLM applications

HUMAN-DRIVEN

1

Code



2

LLM Call



one step only



3

Chain



multiple steps



4

Router



no cycles



AGENT-EXECUTED

5

State Machine



cycles



6

Autonomous



code



LLMs



LangChain

RAG / Gen AI use cases apply to AI Agents!



Operational + Vector Database

No ETL

Consistent data

Reduce complexity
& costs



Retrieval Augmented Generation (RAG)

Personalize LLM
on your data

Cheaper than
fine tuning

Faster iteration
on new data



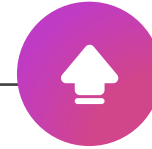
Conversational History

Conversational
context

UX improvements

LLM optimizations

Auditing

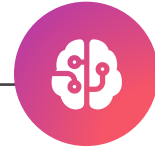


Semantic Caching

Drastically
reduces latency

Saves on Token
consumption

Reduces costs
and latency for LLM



AI Agents

Agent-to-agent
transactions

Statefulness

Logging



Lesson 2 – Cosmos DB search

Vector Search in Azure Cosmos DB for NoSQL

Store data + vectors together

Reduced Complexity & Cost
Transactional Data & Vectors
Optimized for App Developers

Vector Search + Query Filters

Combine with equality,
range & spatial filters
Optimize query focus

Flexible Indexing

Flat, quantized flat, and
DiskANN indexing available

Azure Cosmos DB for NoSQL Capabilities

Serverless or
provisioned
throughput

Built-in
multitenancy

Instant &
dynamic
autoscale

<10ms
point-reads

Globally-
replicated

Industry-leading
99.999% SLA

Vector Index Types in Azure Cosmos DB

No Index

Vectors are stored in the documents (not the index)

100% Accuracy

Not recommended for Prod (expensive/slow)

Useful to find "ground truth"

Flat / kNN

Vectors stored on bw-tree index for better performance

100% Accuracy

Faster than no index

Not recommended for Prod (expensive)

Useful to find "ground truth"

Quantized Flat

Vectors are compressed, then stored on bw-tree index for fast performance

95-100% Accuracy

Good for ~50k vectors and under

DiskANN

Compressed vectors in RAM, uncompressed graph on High-speed SSDs

High accuracy & tunable

Good for larger scales

Microsoft DiskANN in Azure Cosmos DB

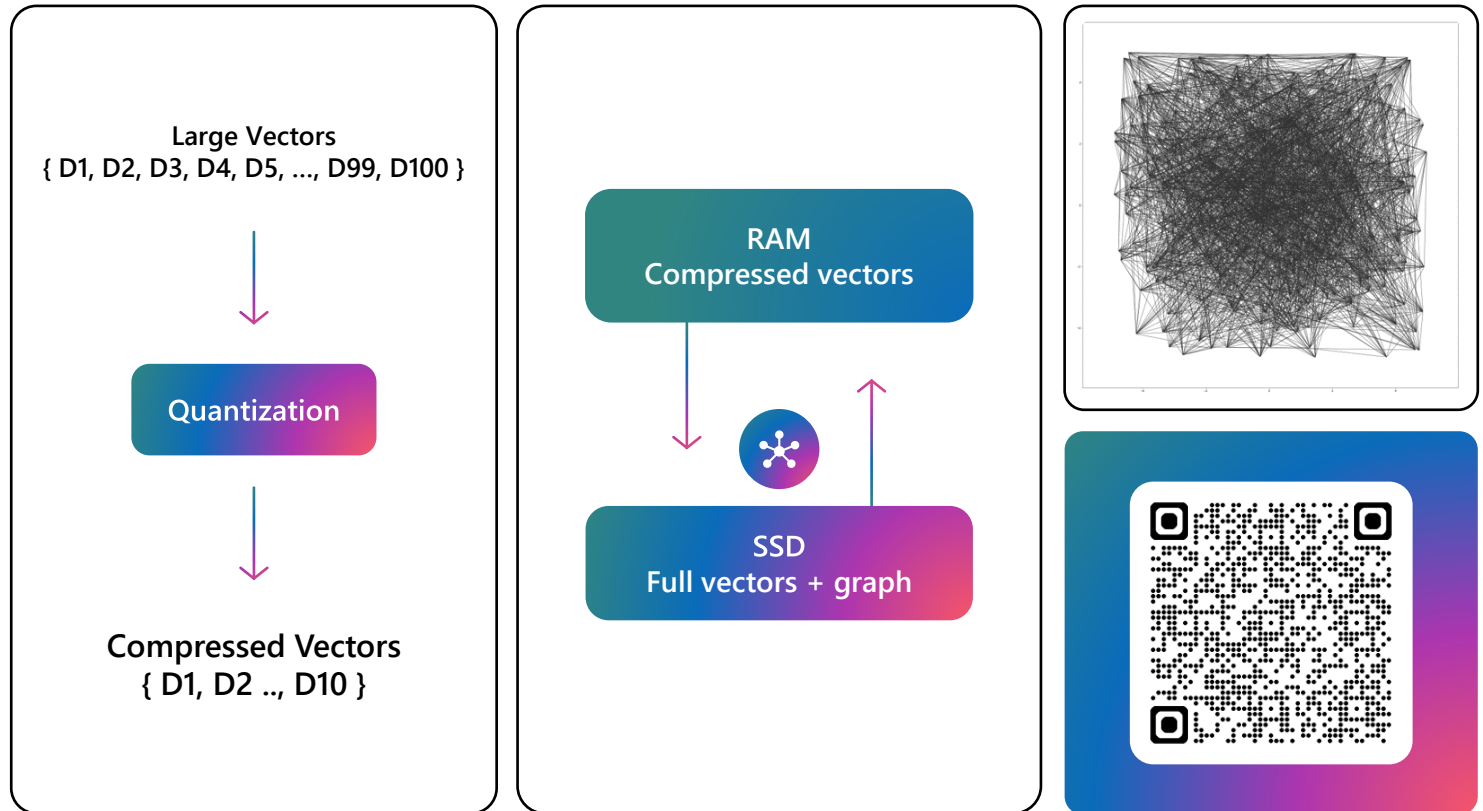
Cost-effective, low latency
vector search at any scale

Unlimited scale

Low latency

Robust to data changes

Serverless



Read the whitepaper: aka.ms/DiskANNCosmosDBWhitePaper

What's new in Vector Search?



Performance improvements

Lower latency vector searches at scale
(<40ms on 100M vectors)



Configurable vector indexes and search

Tune your own tradeoff between latency and accuracy



New compatibilities

Customer managed keys, Point-in-time-restore, Continuous backup, and Fabric mirroring



Modifiable policies

Drop and recreate vector policies and indexes for greater flexibility

* Available March 2025

Preview

Full Text Search & Ranking

Search for text in documents

Improved search relevancy and efficiency with a new Full Text Index

Rank text search results by relevancy with BM25



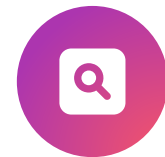
Full Text Search

Quickly and efficiently find keywords and terms in properties



Text Analyzers

- Tokenization
 - Stopword removal
 - Stemming
-



Text Ranking with BM25

Find the documents most relevant to your search

Preview

Hybrid Search for improved search relevancy

Reciprocal Rank Fusion (RRF)

Vector
Similarity
Score



Full Text
Scoring
(BM25)



More Relevant
Search Results

Example Query

```
SELECT TOP 10 *  
FROM c  
ORDER BY RANK  
RRF(FullTextScore(c.text, ['keyword1', 'keyword2']),  
VectorDistance(c.vector, @vector))
```


The two Cosmos DBs that support vector search



Vector Policy, Vector Index and Vector Search

Vector Policy

Define Vector Policy on Container
(can have multiple vector policies per container, but only 1 policy per “path”)

```
{
  "vectorEmbeddings": [
    {
      "path": "/vector1",
      "dataType": "float32",
      "distanceFunction": "cosine",
      "dimensions": 1536
    }
  ]
}
```

VECTOR Index

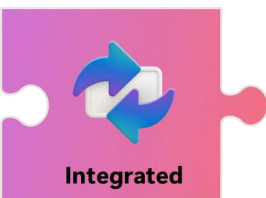
Define vector index type used for each Vector policy
(flat, quantizedFlat, diskANN)

```
{
  "indexingMode": "consistent",
  "automatic": true,
  "includedPaths": [
    {
      "path": "/*"
    }
  ],
  "excludedPaths": [
    {
      "path": "/_etag/?"
    },
    {
      "path": "/vector1/*"
    }
  ],
  "vectorIndexes": [
    {
      "path": "/vector1",
      "type": "diskANN"
    }
  ]
}
```

VECTOR_DISTANCE function

Calculates the distance between two vectors using a specified distance metric , order/rank/page resultset.

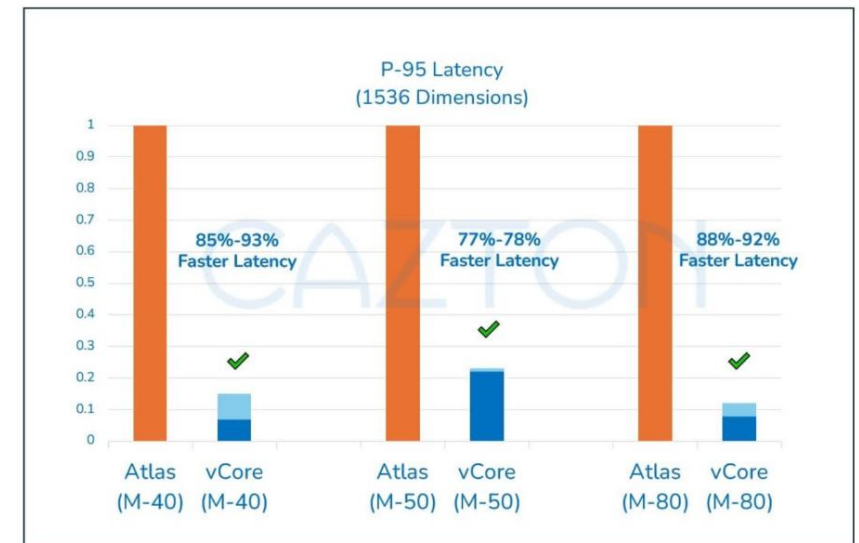
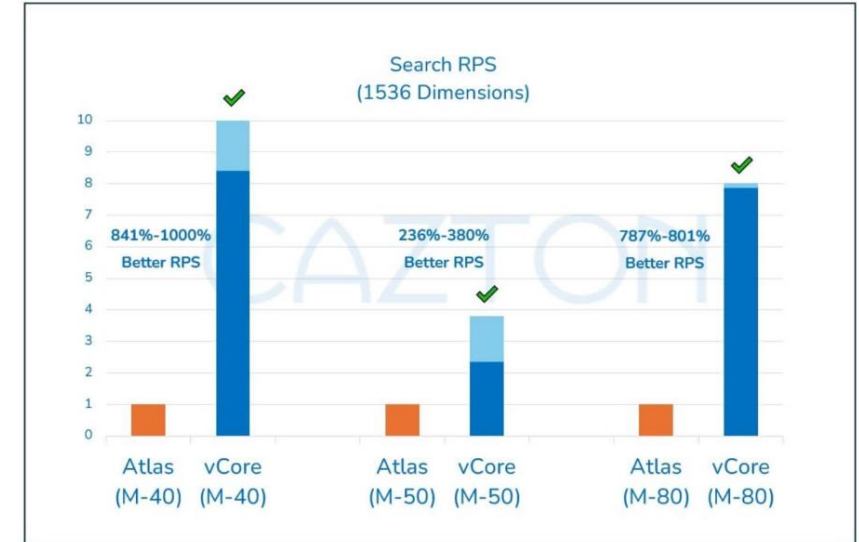
```
SELECT TOP 10 c.title, VectorDistance(c.contentVector,
[1,2,3]) AS SimilarityScore
FROM c
ORDER BY VectorDistance(c.contentVector, [1,2,3])
```



Integrated vector store - Azure Cosmos DB for NoSQL | Microsoft Learn

Cosmos DB for Mongo vCore - AI Investments

Best Vector DB



Lesson 3 – PostgreSQL search

Azure Database for PostgreSQL: Native Vector Search

Enabling Information Retrieval for RAG apps, Semantic Search, and more!

Open-source Pgvector extension provides support to store, index, and query vectors for similarity search scenarios

Supports multiple vector **distance functions**

Enables AI solutions to seamlessly integrate into **existing OLTP Postgres apps** without exporting data to specialize systems

Access control, encryption, high availability, disaster recovery all **“just work”**

Generative AI apps

RAG (Retrieval Augmented Generation) apps

Retrieve private data to ground LLM model responses

Recommendation/Semantic Search

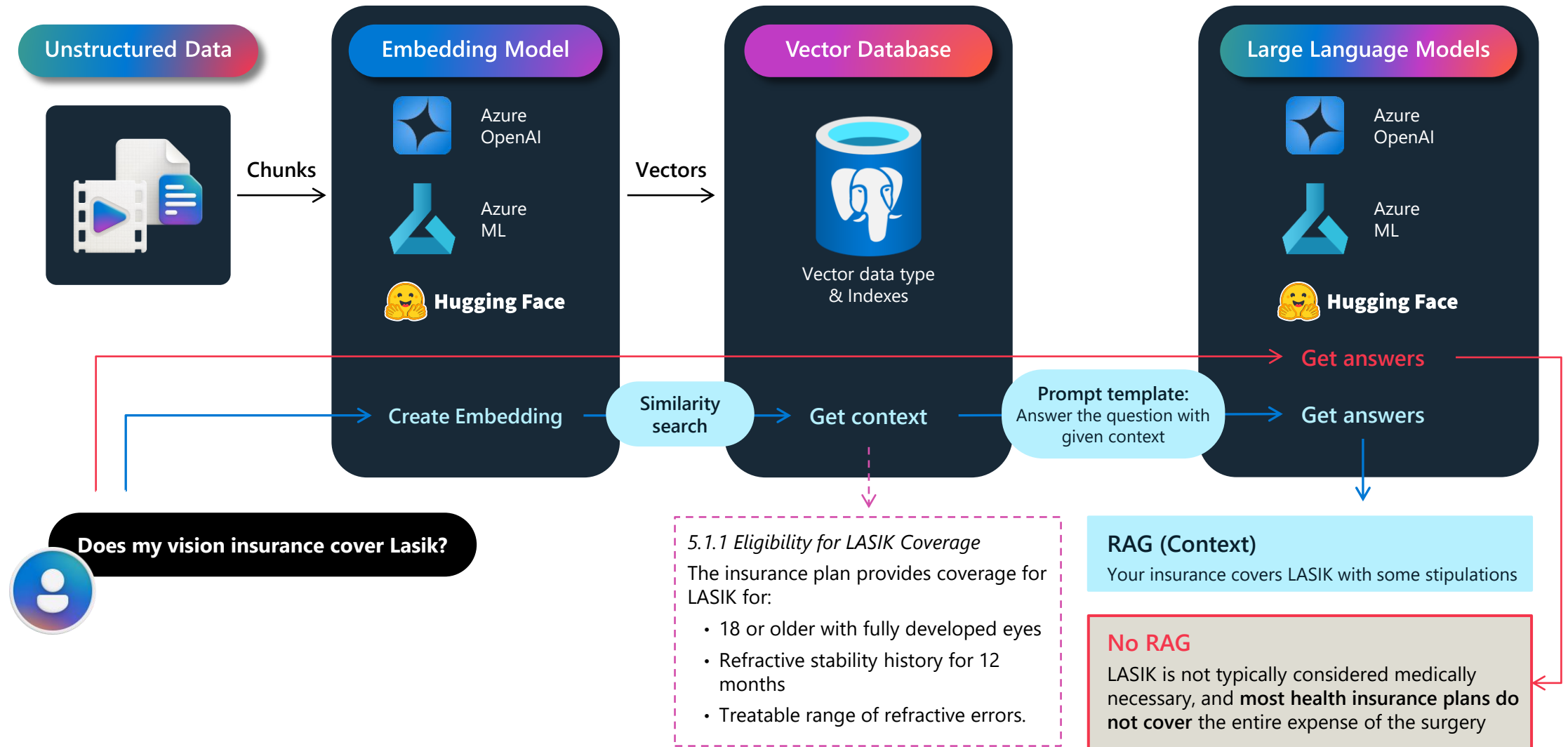
Retrieve similar documents by distance between vectors

Hybrid Search

Combine vector search, row filtering, and full-text search

[Learn more](#)

RAG Pattern with Azure Database for PostgreSQL



Two Problems in Information Retrieval



Problem #1: Scale

Efficiently scaling vector stores to 1M+ of vectors is hard.

Solution

Enhance Scale with Vector Indexing



Problem #2: Accuracy

Quality of GenAI agent responses and vector search accuracy need to improve.

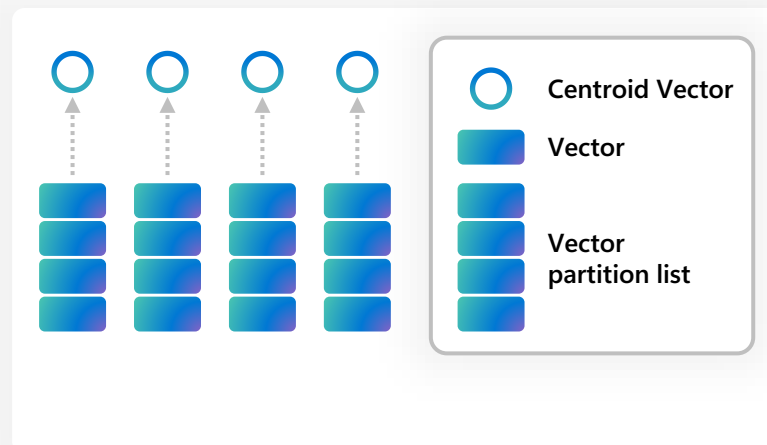
Solution

Improve Accuracy with Advanced RAG architectures

Vector indexes supported by Azure PostgreSQL today

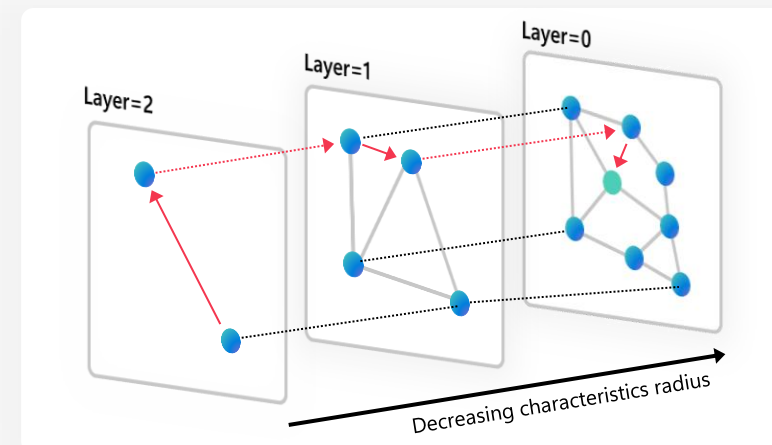
IVFFlat (Inverted File Index)

- Partitions vectors into clusters and assigns each vector to one cluster.
- **Building the index is fast and memory-efficient**
- Requires a separate clustering step before indexing (slow)
- **Tuning parameters is important.** Can be very accurate if configured properly



HNSW (Hierarchical Navigable Small World)

- Builds a multi-layer graph with long and short connections between the vectors.
- **Robust and accurate at scale**
- No-preprocessing step.
- **Can support many inserts/deletes efficiently.**
- **Larger memory footprint**
- It also has many parameters (such as the number of layers and neighbors) that need to be tuned carefully.



When to use IVFFlat or HNSW?

IVFFlat (Inverted File Index)

- More suitable for scenarios where vectors have **low to medium dimensions** (e.g., less than 100), and the similarity metric is Euclidean distance or cosine similarity.
- More suitable for resource constrained systems as memory usage can be at least half that of HNSW.
- Good for static datasets (not updated frequently)

→ Larger datasets with low dimensionality.

HNSW (Hierarchical Navigable Small World)

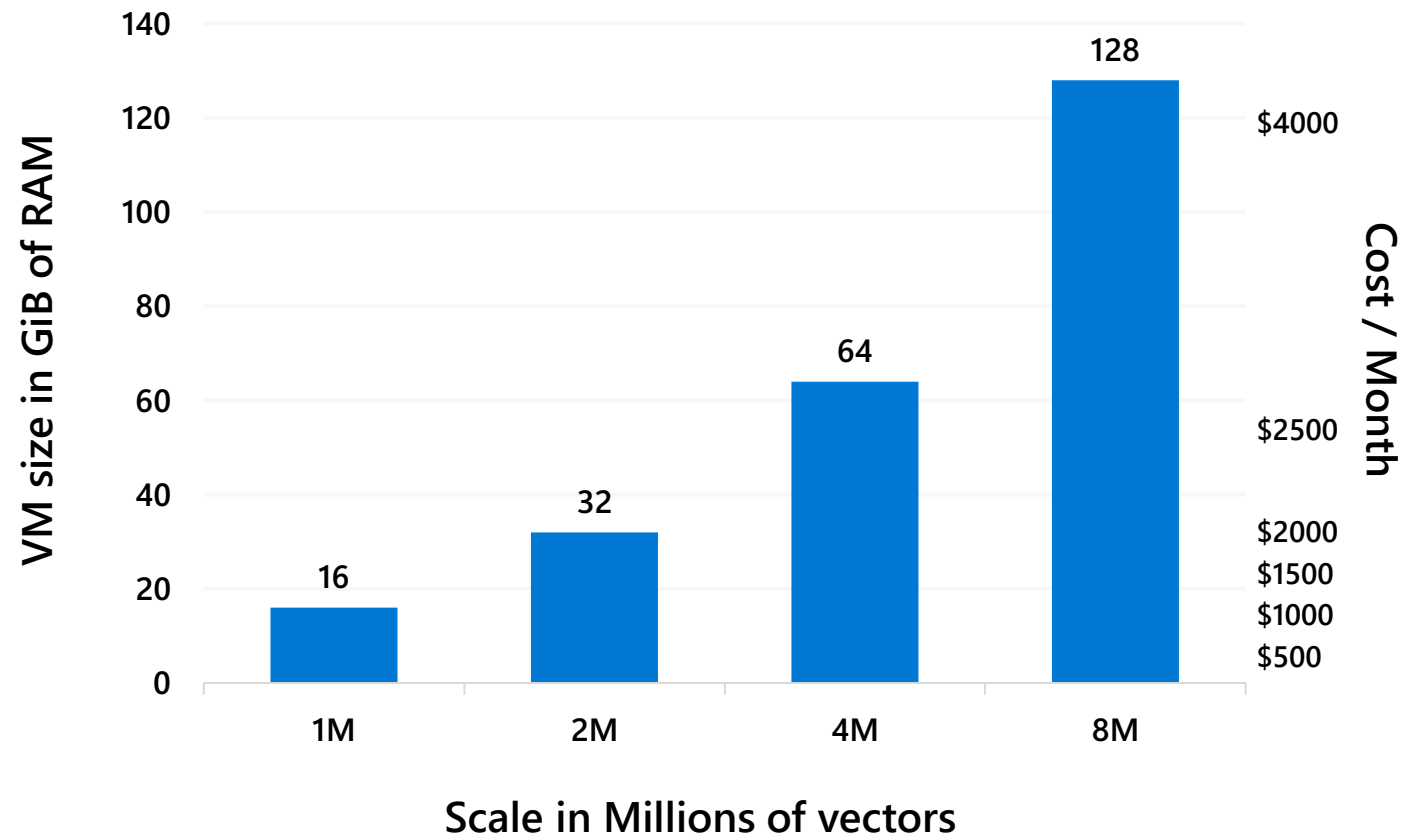
- More suitable for scenarios where vectors have **high dimensions** (e.g., more than 100), and the similarity metric is inner product or angular distance.
- HNSW indexes can be created immediately after table creation. New data inserted into the table is automatically indexed.
- Excellent performance for use cases with up to 2,000 dimensions.

→ Smaller datasets with high dimensionality

Large datasets with high dimensionality : DiskANN Vector Index

Vector store cost grows quickly

Cost of storing full 1536-dimensional embeddings on Postgres (HNSW)





DiskANN Vector Index

Now Generally Available

Highly **performant**, **scalable**, and **accurate** index for vectors

Superior to native PGVector **IVFLAT** & **HNSW** index types

Reduced memory footprint by storing vectors on SSD

Huge **cost savings** at scale due to reduced memory footprint.

Quantization **improves speed** and **accuracy** of vector search

Vector compression

Large Vectors
{ D1, D2, D3, D4, D5, ..., D99, D100 }



Quantization



Compressed Vectors
{ D1, D2 ..., D10 }

Optimized retrieval

RAM
Compressed vectors

Optimized for
minimal SSD reads

SSD
Full vectors + graph



Achieve greater performance and cost savings with DiskANN

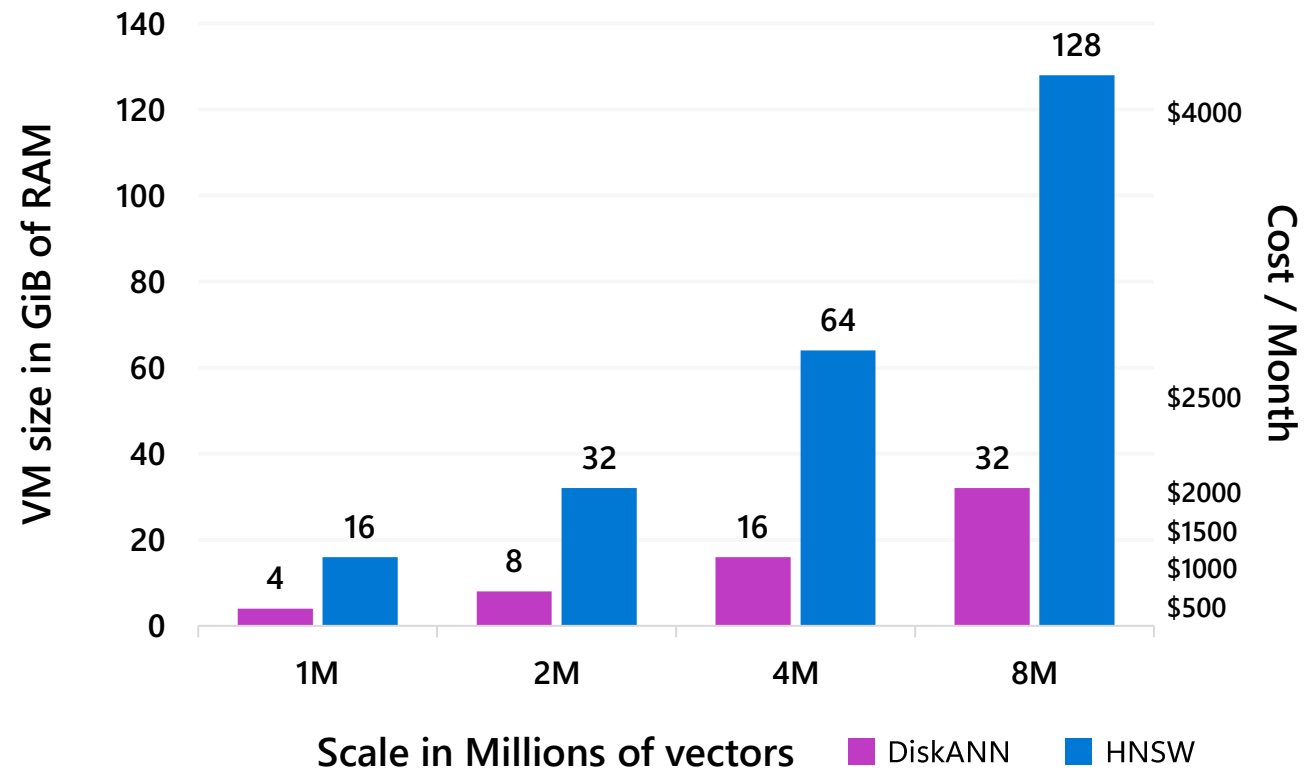


Dataset: Wiki Cohere 35M vectors, 768 dimensions, 236GiB | 95% recall
Compared to HNSW standard configuration



Achieve greater performance and cost savings with DiskANN

Up to
4x
Lower cost
across all VM SKUs



DiskANN Product Quantization – Cost advantage increases at larger scale

[Learn more](#)

Improving Filtered Results on Search

We implemented a filtered result iterator to improve the accuracy of DiskANN.

Query: Top 3 results by similarity AND price < \$30 on 2025-01-02

Results by Ordered Similarity

40 most similar rows will be returned for filtering.

Listing #1 | \$100

Listing #2 | \$40

⋮

Listing #40 | \$50

Listing #41 | \$20

Listing #42 | \$10

Listing #43 | \$10

Filtered with price and date

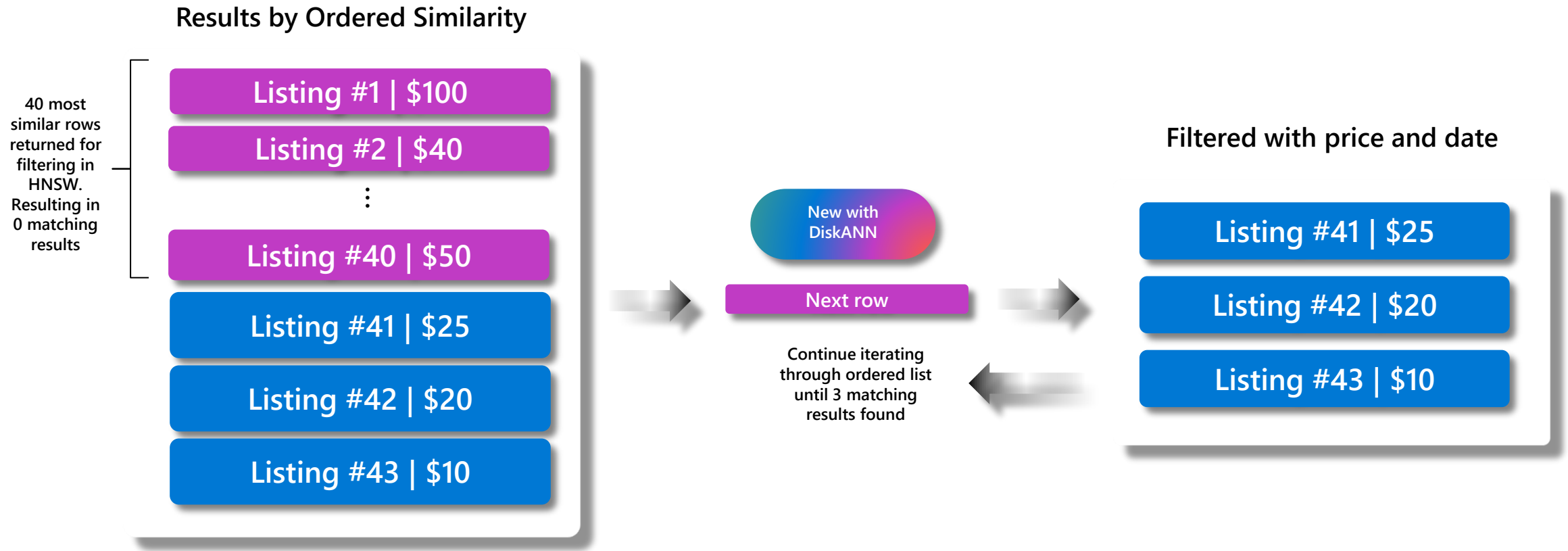
No Results

No rows in 40 most similar rows that price < \$30 on 2025-01-02

Improving Filtered Results on Search

We implemented a filtered result iterator to improve the accuracy of DiskANN.

Query: Top 3 results by similarity AND price < \$30 on 2025-01-02



Improve Accuracy with Advanced RAG architectures

Accuracy improvement strategies in Basic RAG architectures

- Chunking strategies
- Bigger embeddings
- Query rewriting
- Hybrid search
- Metadata filtering

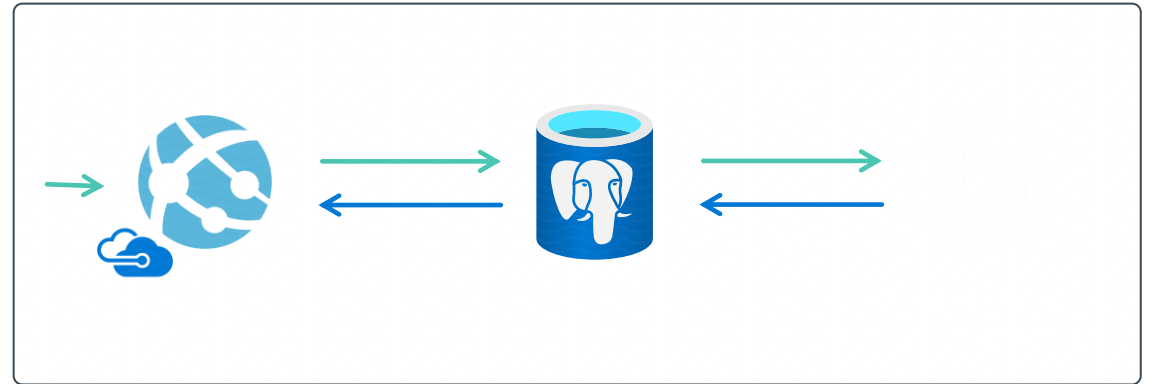
Accuracy improvement strategies in Advanced RAG architectures

- Semantic Ranking
- Hierarchical summarization (RAPTOR)
- Knowledge graphs (GraphRAG)
- Agentic systems

Vector Generation

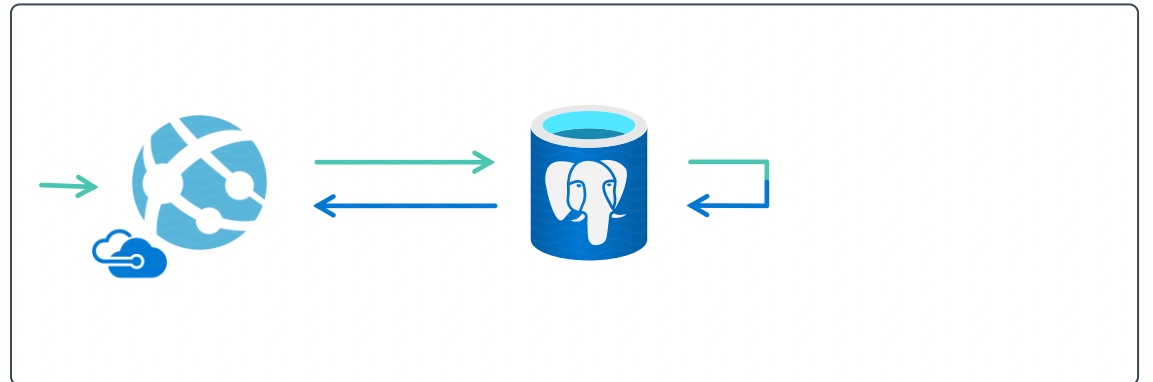
Remote Embedding Models

```
SELECT * FROM <table>
ORDER BY
database_description <->
azure_openai.create_embeddings(
'text-embedding-ada-002',
'Databases with vector support')
```



In-Database Embedding Models (Preview)

```
SELECT * FROM <table>
ORDER BY
recipe_embedding <#>
azure_local_ai.create_embeddings(
'multilingual-e5-small:v1',
'Databases with vector support')
```

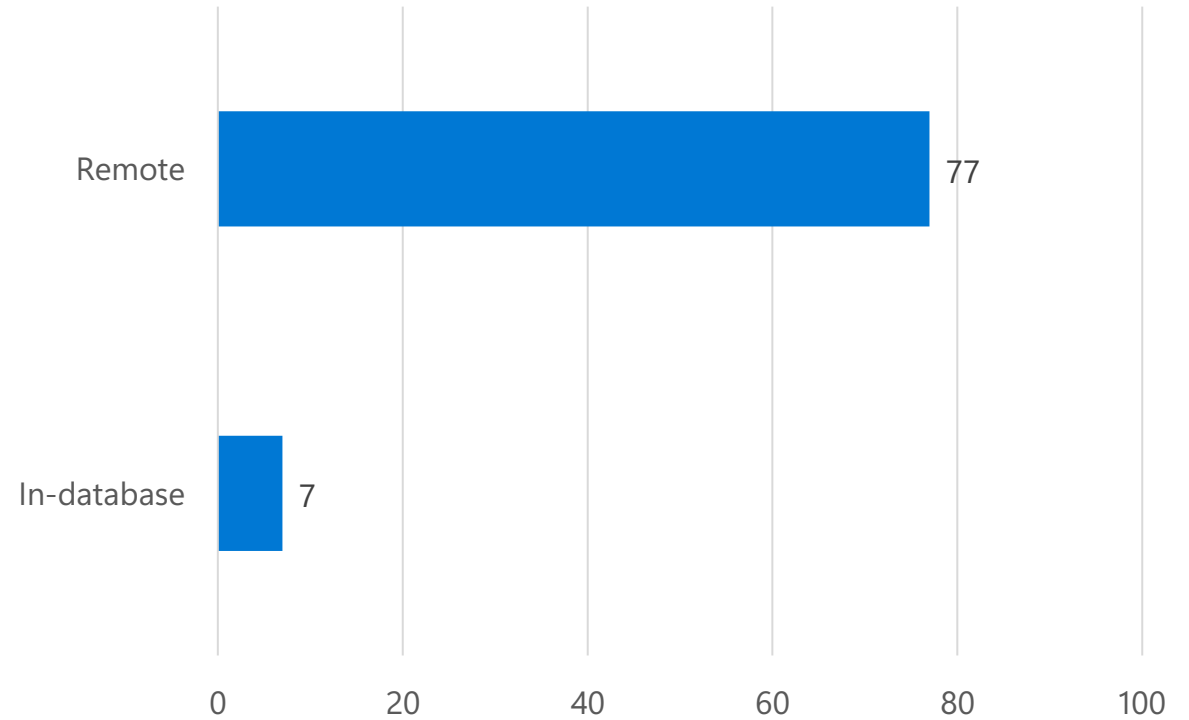


In-Database Embedding Models

Data Privacy, data doesn't leave database

Lower latency and higher throughput

Cost predictability



Search Pattern - ILIKE

 SELECT id, name, opinion Untitled-2 ●

```
1  SELECT id, name, opinion
2  FROM cases
3  WHERE opinion ILIKE '%Water leaking into the apartment from the floor above';
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

AZURE

POSTGRESQL QUERY RESULTS

Results

Messages

id	↑↓ ∇	name	↑↓ ∇	opinion	↑↓ ∇
----	------	------	------	---------	------

Search Phrase – FULL TEXT

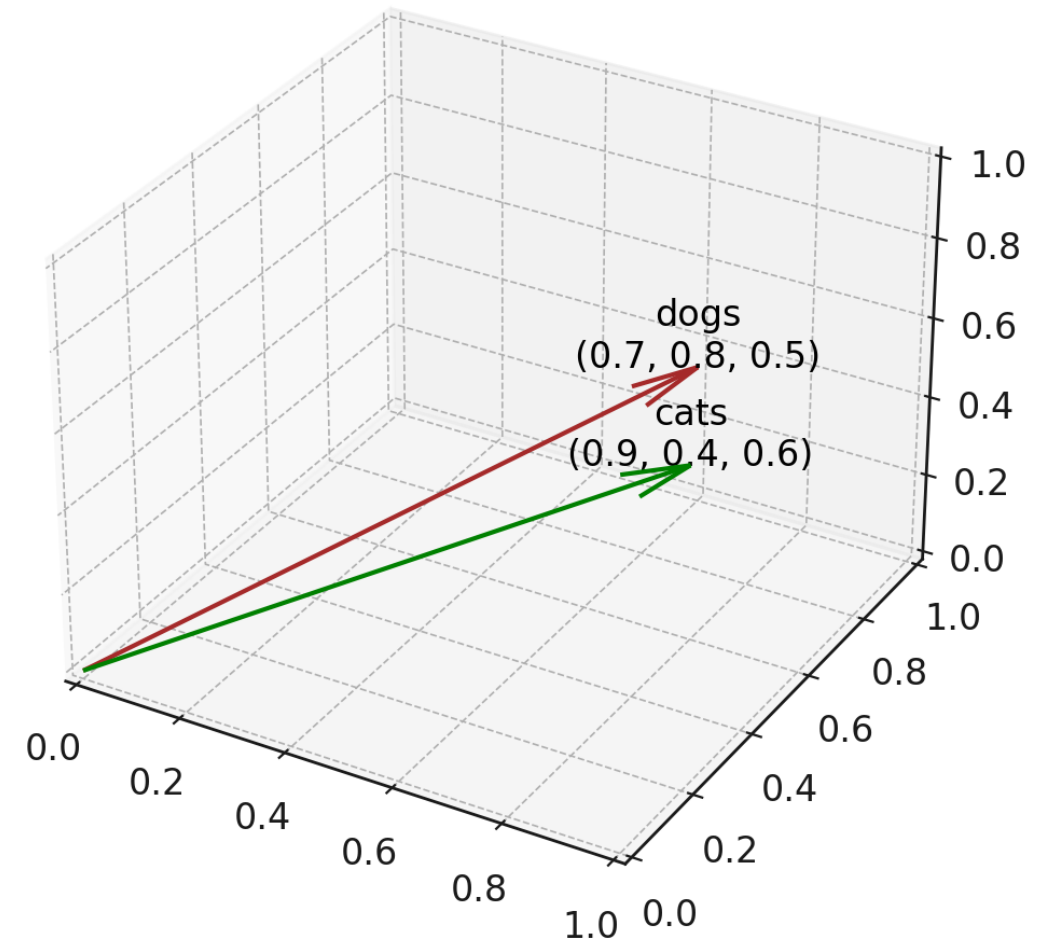
```
1 ALTER TABLE cases
2 ADD COLUMN textsearch tsvector
3 GENERATED ALWAYS AS (to_tsvector('english', name || LEFT(opinion, 8000))) STORED;
4
5 SELECT id, name, opinion
6 FROM cases
7 WHERE textsearch @@ websearch_to_tsquery('Water leaking into the apartment from the floor above.');
```

Results Messages

id	↑↓	🔍	name	↑↓	🔍	opinion	↑↓	🔍
4237124			Pham v. Corbett			"Spearman, C.J.\n¶1 Landlord Lang Pham brought this u...		

Vector 101

- Lists of numbers that represent items in a high-dimensional space.
- For example, a vector representing the string "dogs" might be $[0.7, 0.8, 0.5]$.
- Each number in the vector is a dimension of the space.



Generating vectors

Use a model to generate vectors for items:

Input	→ Model	→ Vector
"dog"	word2vec	[0.017198, -0.007493, -0.057982, ..]
"cat"	word2vec	[0.004059, 0.06719, -0.093874, ...]

Model	Input types	Dimensions
Word2Vec	Word	50-300
OpenAI text-embedding-3	Text	256-3072
Cohere embed-v4.0	Text	256-1536
Azure Computer Vision Multi-modal	Text or Image	1024

Popular models (find more on [HuggingFace](#))

Storing vectors in Postgres table

	name	↕	opinions_vector	↕
1	Le Vette v. Hardman Estate		<code>[-0.0016331548,0.061744377,0.009570962,0.035774387,-0.0054566152,0.025488522...</code>	
2	Uhl Bros. v. Hull		<code>[0.011663684,0.07727938,0.013002017,0.03337608,0.012336588,-0.0059552076,-0....</code>	
3	Woolworth Co. v. City of Seattle		<code>[0.010754321,0.033327676,0.0031798491,0.037272118,0.005627261,3.528203e-05,-...</code>	
4	Stevens v. King County		<code>[0.003271023,0.02641315,0.052163452,0.04789815,-0.012983223,-0.0027828913,-0...</code>	
5	Wilkening v. Watkins Distributors, Inc.		<code>[-0.0191751,0.074948214,0.028303407,0.035043657,-0.036456708,0.025124041,0.0...</code>	
6	Tombari v. City of Spokane		<code>[0.013072358,0.0324271,0.008165626,0.063088655,0.0021425574,0.0009784038,-0....</code>	
7	Downie v. City of Renton		<code>[0.031188406,0.055722144,0.019521315,0.056150556,-0.008168392,-0.010131948,-...</code>	
8	Friskin v. Art Strand Floor Coverings, Inc.		<code>[0.0037472202,0.041518908,-0.0063951095,0.04394401,0.011329315,-0.0045397608...</code>	
9	Imperial Candy Co. v. City of Seattle		<code>[0.017007183,0.03166547,0.022344228,0.04451006,-0.017914034,0.01880602,-0.03...</code>	
10	City of Spokane v. Fisher		<code>[0.0014129812,0.03918049,0.00015752178,0.038623422,0.024062268,0.008920834,-...</code>	

-- Add Embeddings

```
ALTER TABLE cases ADD COLUMN opinions_vector vector(1536);
```

```
UPDATE cases
```

```
SET opinions_vector = azure_openai.create_embeddings('text-embedding-3-small',  
name || opinion, max_attempts => 5, retry_delay_ms => 500)::vector
```

```
WHERE opinions_vector IS NULL;
```

Storing vectors in Postgres table

	name	↑↓🔍	opinions_vector	↑↓🔍
1	Le Vette v. Hardman Estate		<code>[-0.0016331548,0.061744377,0.009570962,0.035774387,-0.0054566152,0.025488522...</code>	
2	Uhl Bros. v. Hull		<code>[0.011663684,0.07727938,0.013002017,0.03337608,0.012336588,-0.0059552076,-0....</code>	
3	Woolworth Co. v. City of Seattle		<code>[0.010754321,0.033327676,0.0031798491,0.037272118,0.005627261,3.528203e-05,-...</code>	
4	Stevens v. King County		<code>[0.003271023,0.02641315,0.052163452,0.04789815,-0.012983223,-0.0027828913,-0...</code>	
5	Wilkening v. Watkins Distributors, Inc.		<code>[-0.0191751,0.074948214,0.028303407,0.035043657,-0.036456708,0.025124041,0.0...</code>	
6	Tombari v. City of Spokane		<code>[0.013072358,0.0324271,0.008165626,0.063088655,0.0021425574,0.0009784038,-0....</code>	
7	Downie v. City of Renton		<code>[0.031188406,0.055722144,0.019521315,0.056150556,-0.008168392,-0.010131948,-...</code>	
8	Frisken v. Art Strand Floor Coverings, Inc.		<code>[0.0037472202,0.041518908,-0.0063951095,0.04394401,0.011329315,-0.0045397608...</code>	
9	Imperial Candy Co. v. City of Seattle		<code>[0.017007183,0.03166547,0.022344228,0.04451006,-0.017914034,0.01880602,-0.03...</code>	
10	City of Spokane v. Fisher		<code>[0.0014129812,0.03918049,0.00015752178,0.038623422,0.024062268,0.008920834,-...</code>	

-- Add Embeddings

```
ALTER TABLE cases ADD COLUMN opinions_vector vector(1536);
```

```
UPDATE cases
```

```
SET opinions_vector = azure_openai.create_embeddings('text-embedding-3-small',  
name || opinion, max_attempts => 5, retry_delay_ms => 500)::vector
```

```
WHERE opinions_vector IS NULL;
```


Search Phrase – Vector Search

ALTER TABLE cases ADD COLUMN opinions_ve

Untitled-2

```
1 ALTER TABLE cases ADD COLUMN opinions_vector vector(1536);
2 UPDATE cases
3 SET opinions_vector = azure_openai.create_embeddings('text-embedding-3-small', name || opinion, max_attempts => 5, ret
4 WHERE opinions_vector IS NULL;
5
6 SELECT
7   id, name, opinion
8 FROM
9   cases
10 ORDER BY opinions_vector <=> azure_openai.create_embeddings('text-embedding-3-small', 'Water leaking into the apartmen
11 LIMIT 10;
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

AZURE

POSTGRESQL QUERY RESULTS

SPELL CHECKER

Results

Messages

Open in New Tab

	id	name	opinion
1	615468	Le Vette v. Hardman Estate	"Morris, J.\nAppeal from an order of nonsuit and dism...
2	768356	Uhl Bros. v. Hull	"Holcomb, J.\nAppellant, subtenant of a former lessee...
3	674990	Woolworth Co. v. City of Seattle	"Mitchell, J.\nAppellant, plaintiff below, was engage...
4	4938756	Stevens v. King County	"Donworth, J.\nRespondents, as plaintiffs, brought th...
5	8848167	Wilkening v. Watkins Distributors, Inc.	"[As amended by order of the Court of Appeals October...
6	1346648	Tombari v. City of Spokane	"Millard, J.\nThis action was brought to recover for ...
7	838633	Downie v. City of Renton	"Millaed, J.\n-This action was brought to enjoin the ...
8	5041745	Frisken v. Art Strand Floor Coverings, Inc.	"Rosellini, J.\nThe respondent, Florence Frisken, is ...
9	630224	Imperial Candy Co. v. City of Seattle	"Main, J.\nThe purpose of this action was to recover ...
10	685636	City of Spokane v. Fisher	"Tolman, J.\nThe city of Spokane was made defendant i...

Create DiskANN vector index

```
CREATE EXTENSION IF NOT EXISTS pg_diskann CASCADE;  
CREATE INDEX my_table_embedding_diskann_idx ON my_table USING diskann (embedding vector_cosine_ops);
```

To use the DiskANN algorithm with pgvector in PostgreSQL you need to enable the **pg_diskann** extension.

The statement above creates an index '[my_table_embedding_diskann_idx](#)' on table '[my_table](#)' using the DiskANN algorithm and using the cosine similarity measurement.

Example of vector search in PostgreSQL:

```
SELECT * FROM my_table ORDER BY embedding <=> @query_vector LIMIT 10;
```

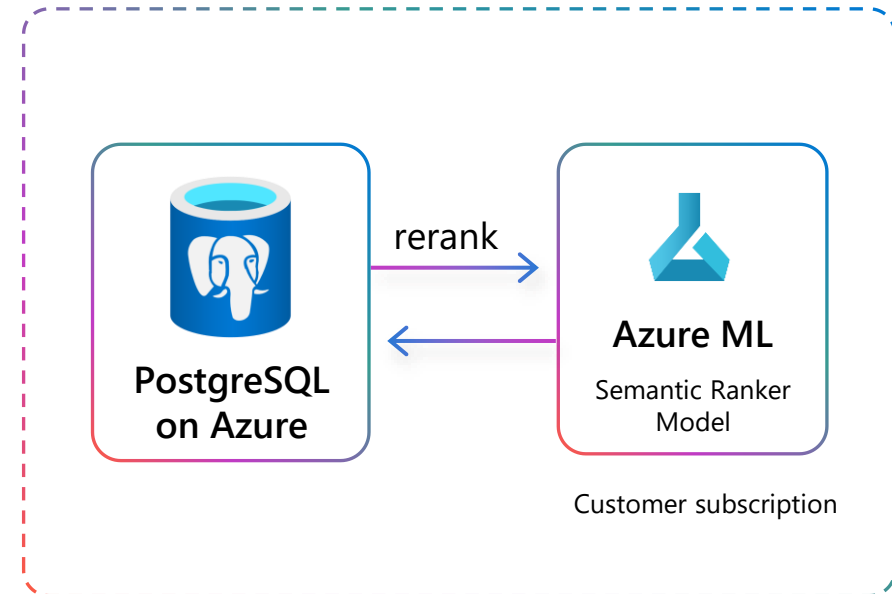
Semantic Ranker Solution Accelerator for Postgres

Overview

- Automated Deployment Script
- SQL Integration

Available Now!

- Blog: aka.ms/pg-ranker
- Repo: aka.ms/pg-ranker-repo



GraphRAG Solution Accelerator for Postgres

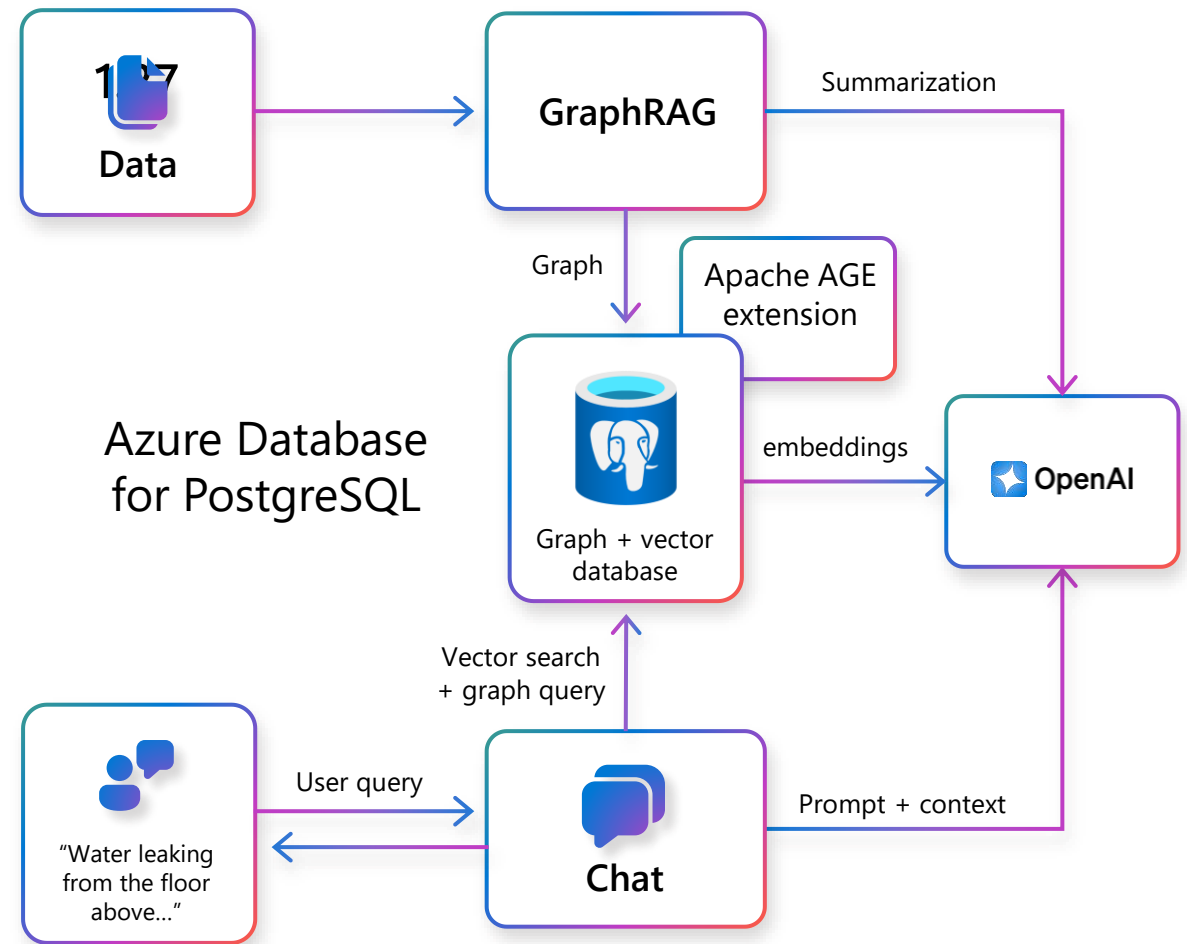
Preview

Overview

- Legal Research Copilot app
- U.S. Case Law dataset (0.5 million cases)

Available Now!

- Blog: aka.ms/pg-graphrag
- Repo: aka.ms/pg-graphrag-repo



Lesson 3 – SQL DB search

Why SQL for AI apps?



45T+

queries run every month

13M+

active databases every
month

60+

regions globally
supported

The developer's intelligent cloud database service

Built for application developers

Gives developers more time to innovate

Accelerates time to market

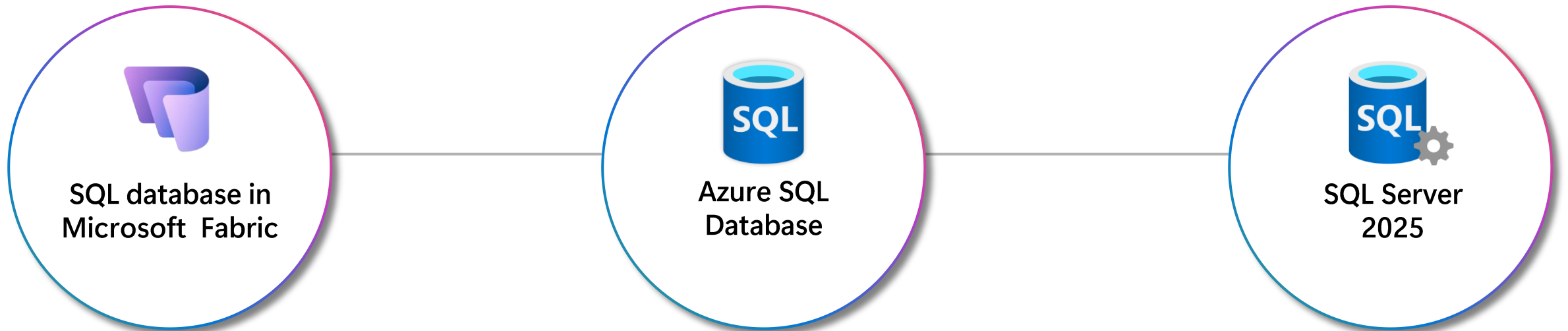
Maximizes performance, reliability,
and data protection

Allows developers to use the languages
and platforms they prefer

Everything built-in



Vector Search



Build Enterprise AI-ready applications ground to cloud

Build RAG patterns inside
the engine



Vector Store

Native vector data type and DiskANN index (coming soon)



Model Management

Model interfaces ground to cloud using REST



Embeddings built-in

Text Chunking and built-in embedding generation



Simple semantic searching

Vector distance and Vector search



Framework integration

LangChain, Semantic Kernel, Entity Framework Core

AOAI via REST, Vector Data Type and Distance Function

Generate Embedding (AOAI REST)

Call AOAI models via REST endpoints from Azure SQL Database using the `sp_invoke_external_rest_endpoint` system stored proc

VECTOR type

VECTOR_DISTANCE function

```
DECLARE @v1 VECTOR(2) = CAST('[10,50]' AS VECTOR(2))  
DECLARE @v2 VECTOR(2) = CAST('[10,50]' AS VECTOR(2))  
SELECT  
    VECTOR_DISTANCE('euclidean', @v1, @v2) AS euclidean,  
    VECTOR_DISTANCE('cosine', @v1, @v2) AS cosine,  
    VECTOR_DISTANCE('dot', @v1, @v2) AS negative_dot_product;
```

EAP for Vector Support Refresh - Introducing Vector type - Azure SQL Devs' Corner

Announcing EAP for Vector Support in Azure SQL Database - Azure SQL Devs' Corner (microsoft.com)



Lab – Vector Database

Lab Description

- Cosmos DB - How get started with basic string search, full-text search, vector search and hybrid search
- PostgreSQL DB – Compute Vector similarity
- SQL DB – Compute Cosine distance between input vector and stored vector

Questions?